Question 1
Wrong
What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)

Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:
The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Wrong
Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong
Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

Quick Sort with time complexity O(kLogk)

D Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 3 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 4
Wrong

Which of the following is not true about comparison based sorting algorithms?

The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

B Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 5
Wrong

What is time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = n; i > 0; i /= 2)
     for (int j = 0; j < i; j++)
        count += 1;
  return count;
}
```

O(n^2)

B O(nLogn)

O(n)

D O(nLognLogn)

Analysis of Algorithms
Discuss it

Question 5 Explanation:
For a input integer n, the innermost statement of fun() is executed following times. n + n/2 + n/4 + ... 1 So time complexity T(n) can be written as T(n) = O(n + n/2 + n/4 + ... 1) = O(n)
The value of count is also n + n/2 + n/4 + .. + 1

Question 6
Correct

What is the time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = 0; i < n; i++)
     for (int j = i; j > 0; j--)
        count = count + 1;
  return count;
}
```

A Theta (n)

Theta (n^2)

C Theta (n*Logn)

D Theta (nLognLogn)

Analysis of Algorithms
Discuss it

Question 6 Explanation:
The time complexity can be calculated by counting number of times the expression "count = count + 1;" is executed. The expression is executed 0 + 1 + 2 + 3 + 4 + ... + (n-1) times.
Time complexity = Theta(0 + 1 + 2 + 3 + .. + n-1) = Theta (n*(n-1)/2) = Theta(n^2)

Question 7
Correct

The recurrence relation capturing the optimal time of the Tower of Hanoi problem with n discs is. (GATE CS 2012)

A T(n) = 2T(n – 2) + 2

B T(n) = 2T(n – 1) + n

C T(n) = 2T(n/2) + 1

T(n) = 2T(n – 1) + 1

Analysis of Algorithms
Discuss it

Question 7 Explanation:
Following are the steps to follow to solve Tower of Hanoi problem recursively.

```
Let the three pegs be A, B and C. The goal is to move n pegs from A to C.
To move n discs from peg A to peg C:
    move n-1 discs from A to B. This leaves disc n alone on peg A
    move disc n from A to C
    move n?1 discs from B to C so they sit on disc n
```

The recurrence function T(n) for time complexity of the above recursive solution can be written as following. T(n) = 2T(n-1) + 1

Question 8
Wrong

Let w(n) and A(n) denote respectively, the worst case and average case running time of an algorithm executed on an input of size n. which of the following is ALWAYS TRUE? (GATE CS 2012)

(A)

(B)

(C)

(D)

A

B B

C

D D

Analysis of Algorithms
Discuss it

Question 8 Explanation:
The worst case time complexity is always greater than or same as the average case time complexity.

Question 9
Wrong

Which of the following is not O(n^2)?

A (15^10) * n + 12099

B n^1.98

n^3 / (sqrt(n))

(2^20) * n

Analysis of Algorithms
Discuss it

Question 9 Explanation:
The order of growth of option c is $n^{2.5}$ which is higher than n^2.

Question 10
Correct

Which of the given options provides the increasing order of asymptotic complexity of functions f1, f2, f3 and f4?

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

f3, f2, f4, f1

B f3, f2, f1, f4

C f2, f3, f1, f4

D f2, f3, f4, f1

Analysis of Algorithms
Discuss it

Question 10 Explanation:

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

Except f3, all other are exponential. So f3 is definitely first in output. Among remaining, n^(3/2) is next. One way to compare f1 and f4 is to take Log of both functions. Order of growth of Log(f1(n)) is Θ(n) and order of growth of Log(f4(n)) is Θ(Logn * Logn). Since Θ(n) has higher growth than Θ(Logn * Logn), f1(n) grows faster than f4(n). **Following is another way to compare f1 and f4.** Let us compare f4 and f1. Let us take few values to compare

```
n = 32, f1 = 2^32, f4 = 32^5 = 2^25
n = 64, f1 = 2^64, f4 = 64^6 = 2^36
..............
..............
```

Also see http://www.wolframalpha.com/input/?i=2^n+vs+n^%28log+n%29 Thanks to fella26 for suggesting the above explanation.

Question 12

Correct

What is the best time complexity of bubble sort?

A N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 12 Explanation:

The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: [sourcecode language="C"] int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once.

Question 13

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

B NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 13 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 14
Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 14 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 15
Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

N(logN base 3)

B N(logN base 2/3)

C N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 15 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 18
Wrong

The following statement is valid. log(n!) = θ(n log n).

True

False

Analysis of Algorithms
Discuss it

Question 18 Explanation:
Order of growth of $\log n!$ and $n \log n$ is same for large values of n, i.e., $\theta(\log n!) = \theta(n \log n)$. So time complexity of fun() is $\theta(n \log n)$. The expression $\theta(\log n!) = \theta(n \log n)$ can be easily derived from following Stirling's approximation (or Stirling's formula).

$$\log n! = n \log n - n + O(\log(n))$$

Question 19

Question
19
Wrong

What does it mean when we say that an algorithm X is asymptotically more efficient than Y?

A X will be a better choice for all inputs

X will be a better choice for all inputs except possibly small inputs

X will be a better choice for all inputs except possibly large inputs

D Y will be a better choice for small inputs

Analysis of Algorithms
Discuss it

Question 19 Explanation:
In asymptotic analysis we consider growth of algorithm in terms of input size. An algorithm X is said to be asymptotically better than Y if X takes smaller time than y for all input sizes n larger than a value n0 where n0 > 0.

Question 20
Correct

What is the time complexity of Floyd–Warshall algorithm to calculate all pair shortest path in a graph with **n** vertices?

A O(n^2logn)

B Theta(n^2logn)

C Theta(n^4)

Theta(n^3)

Analysis of Algorithms

Discuss it

Question 20 Explanation:
Floyd–Warshall algorithm uses three nested loops to calculate all pair shortest path. So, time complexity is Thete(n^3). Read here for more details.

Question 21
Wrong

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 21 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 22
Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 22 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question 23
Wrong

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

T(n) <= 2T(n/2) + n

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 23 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 26
Wrong

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 26 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 27

Question
31
Wrong

The number of elements that can be sorted in $\Theta(logn)$ time using heap sort is

(A) $\Theta(1)$
(B) $\Theta(\sqrt{logn})$
(C) $\Theta(Logn/(LogLogn))$
(d) $\Theta(Logn)$

A

B B

C

D D

Analysis of Algorithms **Sorting** **HeapSort**
Discuss it

Question 31 Explanation:

Time complexity of Heap Sort is $\Theta(mLogm)$ for m input elements. For m = $\Theta(Logn/(LogLogn))$, the value of $\Theta(m * Logm)$ will be $\Theta([Logn/(LogLogn)] * [Log(Logn/(LogLogn))])$ which will be $\Theta([Logn/(LogLogn)] * [LogLogn - LogLogLogn])$ which is $\Theta(Logn)$

Question 32

Question 33

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**

Discuss it

Question 33 Explanation:

Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 37

Wrong

The minimum number of comparisons required to find the minimum and the maximum of 100 numbers is _____.

148

B 147

146

D 140

Analysis of Algorithms **GATE-CS-2014-(Set-1)**

Discuss it

Question 37 Explanation:

Steps to find minimum and maximum element out of n numbers:

```
1. Pick 2 elements(a, b), compare them. (say a > b)
2. Update min by comparing (min, b)
3. Update max by comparing (max, a)
```

Therefore, we need 3 comparisons for each 2 elements, so total number of required comparisons will be (3n)/2 - 2, because we do not need to update min or max in the very first step. Recurrence relation will be:

T(n) = T(⌈n/2⌉)+T(⌊n/2⌋)+2 = 2T(n/2)+2 = ⌈3n/2⌉-2
By putting the value n=100, (3*100/2)-2 = **148** which is answer.

Question 39
Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

B $O(nLogn)$

$Theta(nLogn)$

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 39 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question 42
Wrong

Two matrices M1 and M2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute M1 × M2 will be

best if A is in row-major, and B is in column- major order

B best if both are in row-major order

C best if both are in column-major order

independent of the storage scheme

Analysis of Algorithms **GATE-CS-2004**
Discuss it

Question 42 Explanation:
This is a trick question. Note that the questions asks about time complexity, not time taken by the program. for time complexity, it doesn't matter how we store array elements, we always need to access same number of elements of M1 and M2 to multiply the matrices. It is always constant or O(1) time to do element access in arrays, the constants may differ for different schemes, but not the time complexity.

Question 46
Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 46 Explanation:
Insertion sort runs in $\Theta(n + f(n))$ time, where f(n) denotes the number of inversion initially present in the array being sorted. Source:
http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 47
Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 47 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 49
Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting $n(\geq 2)$ numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T(n/2) + cn$

$T(n) = T(n-1) + T(0) + cn$

C $T(n) = 2T(n-2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms **Sorting** **GATE-CS-2015 (Set 1)** **QuickSort**
Discuss it

Question 49 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 51
Correct

An unordered list contains n distinct elements. The number of comparisons to find an element

in this list that is neither maximum nor minimum is

A Θ(nlogn)

B Θ(n)

C Θ(logn)

Θ(1)

Analysis of Algorithms **GATE-CS-2015 (Set 2)**
Discuss it

Question 51 Explanation:
We only need to consider any 3 elements and compare them. So the number of comparisons is constants, that makes time complexity as Θ(1) The catch here is, we need to return any element that is neither maximum not minimum. Let us take an array {10, 20, 15, 7, 90}. Output can be 10 or 15 or 20 Pick any three elements from given liar. Let the three elements be 10, 20 and 7. Using 3 comparisons, we can find that the middle element is 10.

Question 53
Correct

In the following table, the left column contains the names of standard graph algorithms and the right column contains the time complexities of the algorithms. Match each algorithm with its time complexity.

1. Bellman-Ford algorithm 2. Kruskal's algorithm 3. Floyd-Warshall algorithm 4. Topological sorting	A : O (m log n) B : O (n^3) C : O (nm) D : O (n + m)

1→ C, 2 → A, 3 → B, 4 → D

B 1→ B, 2 → D, 3 → C, 4 → A

C 1→ C, 2 → D, 3 → A, 4 → B

D 1→ B, 2 → A, 3 → C, 4 → D

Analysis of Algorithms **Gate IT 2005**
Discuss it

Question 53 Explanation:

- **Bellman-Ford algorithm: Time complexity: O(VE)**
- **Kruskal's algorithm**:**Time Complexity:** O(ElogE) or O(ElogV). Sorting of edges takes O(ELogE) time. After sorting, we iterate through all edges and apply find-union algorithm. The find and union operations can take atmost O(LogV) time. So overall complexity is O(ELogE + ELogV) time. The value of E can be atmost V^2, so O(LogV) are O(LogE) same. Therefore, overall time complexity is O(ElogE) or O(ElogV)
- **Floyd-Warshall algorithm**: **Time Complexity**: O(V^3)
- **Topological sorting: Time Complexity:** The above algorithm is simply DFS with an extra stack. So time complexity is same as DFS which is O(V+E).

Question 55
Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 1)**
Discuss it

Question 55 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and recursive formula becomes $T(n) = T(n-1) + \Theta(n)$. An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 56
Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 56 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in $\Theta(n^2)$ time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in $\Theta(n)$ time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by

following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity $\Theta(n^2)$ [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 57

Question 58

Correct

The characters a to h have the set of frequencies based on the first 8 Fibonacci numbers as follows

a : 1, b : 1, c : 2, d : 3, e : 5, f : 8, g : 13, h : 21/

A Huffman code is used to represent the characters. What is the sequence of characters corresponding to the following code? 110111100111010

fdheg

B ecgdf

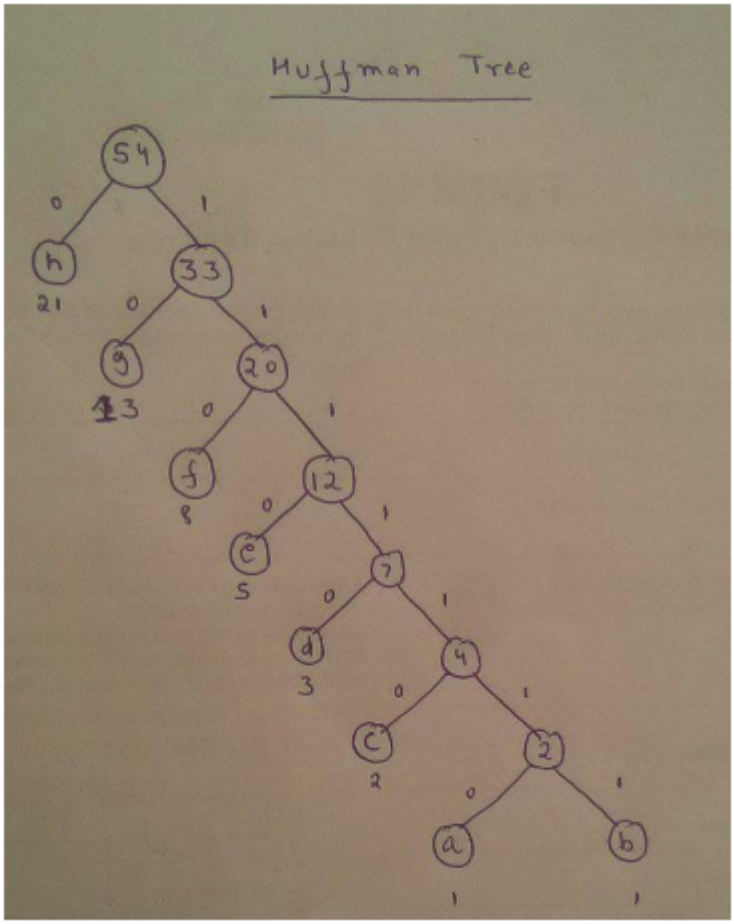
C dchfg

D fehdg

Analysis of Algorithms **GATE IT 2006**
Discuss it

Question 58 Explanation:

Background Required - Generating Prefix codes using Huffman Coding. First we apply greedy algorithm on the frequencies of the characters to generate the binary tree as shown in the Figure given below. Assigning 0 to the left edge and 1 to the right edge, prefix codes for the characters are as below.



a - 1111110 b - 1111111 c - 111110 d - 11110 e - 1110 f - 110 g - 10 h - 0 Given String can be decomposed as 110 11110 0 1110 10 f d h e g This solution is contributed by **Pranjul Ahuja** .

Question 61

Wrong

Which of the following is TRUE?

The cost of searching an AVL tree is θ (log n) but that of a binary search tree is O(n)

B The cost of searching an AVL tree is θ (log n) but that of a complete binary tree is θ (n log n)

The cost of searching a binary search tree is O (log n) but that of an AVL tree is θ(n)

D The cost of searching an AVL tree is θ (n log n) but that of a binary search tree is O(n)

Analysis of Algorithms **Balanced Binary Search Trees** **Gate IT 2008**
Discuss it

Question 61 Explanation:
AVL tree is a balanced tree.
AVL tree's time complexity of searching = θ(logn)
But a binary search tree, may be skewed tree, so in worst case BST searching time = θ(n)

Question 63

Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A $\Theta(n)$

B $\Theta(kn)$

$\Theta(nlogn)$

D $\Theta(n^2)$

Analysis of Algorithms **Sorting** **RadixSort** **Gate IT 2008**
Discuss it

Question 63 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 64
Correct

The auxiliary space of insertion sort is O(1), what does O(1) mean ?

A The memory (space) required to process the data is not constant.

It means the amount of extra memory Insertion Sort consumes doesn't depend on the input. The algorithm should use the same amount of memory for all inputs.

C It takes only 1 kb of memory .

D It is the speed at which the elements are traversed.

Analysis of Algorithms **Sorting** **InsertionSort**
Discuss it

Question 64 Explanation:
The term O(1) states that the space required by the insertion sort is constant i.e., space required doesn't depend on input.

Question 65
Wrong

A sorting technique is called stable if:

A It takes O(nlog n)time

It maintains the relative order of occurrence of non-distinct elements

C It uses divide and conquer paradigm

It takes O(n) space

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 66
Correct

Suppose we want to arrange the ii numbers stored in an array such that all negative values occur before all positive ones. Minimum number of exchanges required in the worst case is:

A n-1

B n

C n+1

None of the above

Analysis of Algorithms **GATE CS 1999**
Discuss it

Question 66 Explanation:
When we have 'i' numbers stored in an array, we have to swap all positive numbers with negative and in worst case positive numbers will be i/2.

Question 67
Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 67 Explanation:
In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 68

Question
71
Wrong

Which of the following algorithm design technique is used in finding all pairs of shortest distances in a graph?

Dynamic programming

B Backtracking

Greedy

D Divide and Conquer

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 72
Wrong

Give the correct matching for the following pairs:

```
A. O(log n)      1. Selection sort
B. O(n)          2. Insertion sort
C. O(nlog n)     3. Binary search
D. O(n^2)        4. Merge sort

codes:
     A   B   C   D
a.   3   1   2   4
```

b. 3 1 4 2
c. 1 3 4 2
d. 1 4 3 2

a

b

C c

D d

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 75
Correct

The correct matching for the following pairs is

(A) All pairs shortest path	(1) Greedy
(B) Quick Sort	(2) Depth-First search
(C) Minimum weight spanning tree	(3) Dynamic Programming
(D) Connected Components	(4) Divide and and Conquer

Codes:

	A	B	C	D
a	2	4	1	3
b	3	4	1	2
c	3	4	2	1
d	4	1	2	3

A a

b

C c

D d

Analysis of Algorithms **GATE CS 1997**
Discuss it

Question 78
Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

(i) 1, 2, 3,....., n
(ii) n, n-1, n-2,....., 2, 1

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 78 Explanation:

Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 83
Correct

Given A, an array of size n, comprised of an increasing sequence of numbers followed immediately by a decreasing one. What is worst case time complexity of optimal algorithm to determine if a given number x is in the array?

Θ(log n)

Θ(n)

C Θ(n^2)

D Θ(log n)^2

Analysis of Algorithms **GATE CS Mock 2018**
Discuss it

Question 83 Explanation:
This is an application of Binary search, which has time complexity Θ(log n) in worst case. Option (A) is correct.

Question 84
Wrong

Match the following:

Algorithm	Time complexity (Worst case)
A. Optimal merge pattern	1. O(m*n)
B. Power of an element	2. O(log n)
C. Prims MST algorithm	3. O(n log n)
D. Longest common subsequence	4. O(m log n)

Which of the following option is correct?

A – 3, B – 2, C – 4, D – 1

A – 3, B – 1, C – 4, D – 2

C A – 3, B – 4, C – 2, D – 1

D A – 1, B – 2, C – 4, D – 3

Analysis of Algorithms **GATE CS Mock 2018 | Set 2**
Discuss it

Question 84 Explanation:
Correct matching is A – 3, B – 2, C – 4, D – 1. Option (A) is correct.

Question 88
Wrong

If b is the branching factor and m is the maximum depth of the search tree, what is the space complexity of greedy search?

A O(b+m)

O(bm)

O(b^m)

D $O(m^m)$

Analysis of Algorithms **UGC-NET CS 2017 Nov - III**
Discuss it

Question 88 Explanation:

1. In binary tree branching factor is 2 and space complexity for height n is $O(2^n)$.
2. In ternary tree branching factor is 3 and space complexity for height n is $O(3^n)$.

   Similarly

3. If branching factor is b and height is m for search tree then space complexity of greedy search is $O(b^m)$.

So, option (C) is correct.

Question 89
Wrong
The time complexity of computing the transitive closure of a binary relation on a set of n elements is known to be

O(nlogn)

B $O(n^{3/2})$

$O(n^3)$

D O(n)

Analysis of Algorithms **Set Theory & Algebra** **ISRO CS 2017 - May**
Discuss it

Question 89 Explanation:
Transitive closure generally uses Floyd-Warshall Algorithm which gives a time complexity of $O(n^3)$ Refer: Transitive closure of a graph

Question 92
Wrong
Any decision tree that sorts n elements has height _____.

Ω(lg n)

B Ω(n)

Ω(n lg n)

D $\Omega(n^2)$

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 93
Correct
Dijkstra's algorithm is based on

A Divide and conquer paradigm

B Dynamic programming

Greedy Approach

D Backtracking paradigm

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 93 Explanation:
Dijkstra's algorithm is based on Greedy Approach. For more information Refer:Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.

Question 94
Correct

Match the following with respect to algorithm paradigms :

	List – I		**List – II**
a.	Merge sort	i.	Dynamic programming
b.	Huffman coding	ii.	Greedy approach
c.	Optimal polygon triangulation	iii.	Divide and conquer
d.	Subset sum problem	iv.	Back tracking

Codes :

	a	b	c	d
(1)	iii	i	ii	iv
(2)	ii	i	iv	iii
(3)	ii	i	iii	iv
(4)	iii	ii	i	iv

A (1)

B (2)

C (3)

(4)

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 95
Correct

What is the best method to go for the game playing problem?

A Optimal Search

B Random Search

Heuristic Search

D Stratified Search

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 97
Correct

Consider a weighted complete graph G on the vertex set $\{v_1, v_2, ... v_n\}$ such that the weight of the edge (v_i, v_j) is $4 | i - j|$. The weight of minimum cost spanning tree of G is :

A $4n^2$

B n

4n – 4

D 2n – 2

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98

Wrong

Match the following:

a.	Prim's algorithm	i.	$O(V^2E)$
b.	Bellman-Ford algorithm	ii.	$O(VE\ lgV)$
c.	Floyd-Warshall algorithm	iii.	$O(E\ lgV)$
d.	Johnson's algorithm	iv.	$O(V^3)$

Where V is the set of nodes and E is the set of edges in the graph.

Codes :

	a	b	c	d
(1)	i	iii	iv	ii
(2)	i	iii	ii	iv
(3)	iii	i	iv	ii
(4)	iii	i	ii	iv

(1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98 Explanation:

- Prim's algorithm takes O(E lgV) time.
- Bellman-Ford algorithm takes $O(V^2E)$ time.
- Floyd-Warshall algorithm takes $O(V^3)$ time.
- Johnson's algorithm takes O(VE lgV) time.

So, option (C) is correct.

Question 99

Correct

(a)	Huffman codes	(i)	$O(n^2)$
(b)	Optimal polygon triangulation	(ii)	$\theta(n^3)$
(c)	Activity selection problem	(iii)	O(nlgn)
(d)	Quicksort	(iv)	$\theta(n)$

Codes :

	(a)	(b)	(c)	(d)
(1)	(i)	(ii)	(iv)	(iii)
(2)	(i)	(iv)	(ii)	(iii)
(3)	(iii)	(ii)	(iv)	(i)
(4)	(iii)	(iv)	(ii)	(i)

Match the following:

A (1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 July – III**

Discuss it

Question 99 Explanation:

- Huffman codes takes O(nlgn) time.
- Optimal polygon triangulation takes $\theta(n^3)$ time

  Activity selection problem takes $\theta(n)$ time

-
- Quicksort takes $O(n^2)$ time

So, option (C) is correct.

Question 100

Correct

Given two sequences X and Y : X = < a, b, c, b, d, a, b > Y = < b, d, c, a, b, a > . The longest common subsequence of X and Y is :

A < b, c, a>

B < c, a, b>

C < b, c, a, a>

< b, c, b, a>

Analysis of Algorithms **UGC NET CS 2015 Dec – III**

Discuss it

Question 101

Wrong

Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Greedy algorithm, $\theta (V^3)$

B Greedy algorithm, $\theta (V^2 \lg n)$

Dynamic programming, θ (V^3)

D Dynamic programming, θ (V^2 lgn)

Analysis of Algorithms **UGC NET CS 2015 Dec – III**
Discuss it

Question 101 Explanation:
Floyd-Warshall algorithm utilizes dynamic programming to solve the all-pairs shortest paths problem on a directed graph in θ (V^3) time. For more information on Floyd-Warshall algorithm ReferDynamic Programming | Set 16 (Floyd Warshall Algorithm) Option (C) is correct.

Question 102
Correct

To determine the efficiency of an algorithm the time factor is measured by:

A Counting micro seconds

Counting number of key operations

C Counting number of statements

D Counting kilobytes of algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - II**
Discuss it

Question 102 Explanation:
To determine the efficiency of an algorithm the time factor is measured by counting number of key operations. It is not measured by counting of statements because less no of statement does not mean that program is more efficient. Many times we have to write large code for better efficieny. We measure space complexity in kilobytes. So, option (B) is correct.

Question 104
Wrong

An all-pairs shortest-paths problem is efficiently solved using:

Dijkstra' algorithm

B Bellman-Ford algorithm

C Kruskal algorithm

Floyd-Warshall algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - III**
Discuss it

Question 104 Explanation:
An all-pairs shortest-paths problem is efficiently solved using Floyd-Warshall algorithm. For more information on all-pairs shortest-paths problem and its solution using Floyd-Warshall algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) So, option (D) is correct.

Match the following:

List - I List - II

(a) Bucket sort	(i) $O(n^3 \lg n)$
(b) Matrix chain multiplication	(ii) $O(n^3)$
(c)Huffman codes	(iii) O(nlgn)
(d)All pairs shortest paths	(iv)O(n)

codes:

	(a)	(b)	(c)	(d)
(A)	(iv)	(ii)	(i)	(iii)
(B)	(ii)	(iv)	(i)	(iii)
(C)	(iv)	(ii)	(i)	(iii)
(D)	(iii)	(ii)	(iv)	(i)

(A)
B (B)
C (C)
(D)

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110
Correct
Dijkstra algorithm, which solves the single-source shortest--paths problem, is a _____, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a _____

A Greedy algorithm, Divide-conquer algorithm
B Divide-conquer algorithm, Greedy algorithm
Greedy algorithm, Dynamic programming algorithm
D Dynamic programming algorithm, Greedy algorithm

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110 Explanation:
Dijkstra algorithm, which solves the single-source shortest–paths problem, is a Greedy algorithm, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a Dynamic programming algorithm. For more information on Floyd-Warshall and Dijkstra algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.
Question 114
Wrong

Match the following:

codes:

List - I

(a) The 8-Queen's problem

(b) Single-Source shortest paths

(c)STRASSEN's Matrix multiplication

(d)Optimal binary search trees

List - II

(i)Dynamic programming

(ii) Divide and conquer

(iii)Greedy approach

(iv)Backtracking

	(a)	(b)	(c)	(d)
(1)	(iv)	(i)	(iii)	(ii)
(2)	(iv)	(iii)	(i)	(ii)
(3)	(iii)	(iv)	(i)	(ii)
(4)	(iv)	(iii)	(ii)	(i)

A (1)
(2)
C (3)
(4)

Analysis of Algorithms **UGC NET CS 2018 July - II**
Discuss it

Question 114 Explanation:

- The 8-Queen's problem is a Backtracking algorithm.
- Single-Source shortest paths is a Greedy approach
- STRASSEN's Matrix multiplication is a Divide and conquer
- Optimal binary search trees is a Dynamic programming. So, option (D) is correct.

Question 1
Correct

What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

A Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)
Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:

The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int

si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Correct
Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

B O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong
Which of the following is not a stable sorting algorithm in its typical implementation.

Insertion Sort

B Merge Sort

Quick Sort

D Bubble Sort

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 3 Explanation:
See following for details. http://www.geeksforgeeks.org/stability-in-sorting-algorithms/
Question 4
Correct
Which of the following sorting algorithms in its typical implementation gives best performance when applied on an array which is sorted or almost sorted (maximum 1 or two elements are misplaced).

A Quick Sort

B Heap Sort

C Merge Sort

Insertion Sort

Sorting **QuickSort** **InsertionSort** **HeapSort**
Discuss it

Question 4 Explanation:
Insertion sort takes linear time when input array is sorted or almost sorted (maximum 1 or 2 elements are misplaced). All other sorting algorithms mentioned above will take more than lienear time in their typical implementation.

Question 5
Wrong

Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

C Quick Sort with time complexity O(kLogk)

Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 5 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 6

Question 6
Wrong

Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

A Heap Sort

Selection Sort

C Insertion Sort

Merge Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 6 Explanation:

Selection sort makes O(n) swaps which is minimum among all sorting algorithms mentioned above.

Question 7
Wrong
Which of the following is not true about comparison based sorting algorithms?

A The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 7 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 8
Wrong
Suppose we are sorting an array of eight integers using quicksort, and we have just finished the first partitioning with the array looking like this:

```
2  5  1  7  9  12  11  10
```

Which statement is correct?

The pivot could be either the 7 or the 9.

B The pivot could be the 7, but it is not the 9

C The pivot is not the 7, but it could be the 9

Neither the 7 nor the 9 is the pivot.

Sorting **QuickSort**
Discuss it

Question 8 Explanation:
7 and 9 both are at their correct positions (as in a sorted array). Also, all elements on left of 7 and 9 are smaller than 7 and 9 respectively and on right are greater than 7 and 9 respectively.

Question 9
Correct
Suppose we are sorting an array of eight integers using heapsort, and we have just finished some heapify (either maxheapify or minheapify) operations. The array now looks like this: 16 14 15 10 12 27 28 How many heapify operations have been performed on root of heap?

A 1

2

C 3 or 4

D 5 or 6

Sorting **Heap** **HeapSort**
Discuss it

Question 9 Explanation:
In Heapsort, we first build a heap, then we do following operations till the heap size becomes 1. a) Swap the root with last element b) Call heapify for root c) reduce the heap size by 1. In this question, it is given that heapify has been called few times and we see that last two elements in given array are the 2 maximum elements in array. So situation is clear, it is maxheapify whic has been called 2 times.

Question 10

Wrong

What is the best time complexity of bubble sort?

N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 10 Explanation:
The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once .

Question 11

Wrong

You have to sort 1 GB of data with only 100 MB of available main memory. Which sorting technique will be most appropriate?

Heap sort

Merge sort

C Quick sort

D Insertion sort

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 11 Explanation:
The data can be sorted using external sorting which uses merging technique. This can be done as follows: 1. Divide the data into 10 groups each of size 100. 2. Sort each group and write them to disk. 3. Load 10 items from each group into main memory. 4. Output the smallest item from the main memory to disk. Load the next item from the group whose item was chosen. 5. Loop step #4 until all items are not outputted. The step 3-5 is called as merging technique.

Question 12

Wrong

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 12 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 13

Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 13 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 14

Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

A N(logN base 3)

B N(logN base 2/3)

N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 14 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 15

Correct

Which sorting algorithm will take least time when all elements of input array are identical? Consider typical implementations of sorting algorithms.

Insertion Sort

B Heap Sort

C Merge Sort

D Selection Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 15 Explanation:
The insertion sort will take θ(n) time when input array is already sorted.

Question 16

Correct

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

A O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 16 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 17

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 17 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question
18

Question 18

Correct

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

D T(n) <= 2T(n/2) + n

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 18 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there

are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 19

Correct

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

B Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 19 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 20

Correct

Which sorting algorithms is most efficient to sort string consisting of ASCII characters?

A Quick sort

B Heap sort

C Merge sort

Counting sort

Sorting **QuickSort** **HeapSort** **CountingSort**
Discuss it

Question 20 Explanation:
Counting sort algorithm is efficient when range of data to be sorted is fixed. In the above question, the range is from 0 to 255(ASCII range). Counting sort uses an extra constant space proportional to range of data.

Question 22

Correct

Which of the following is true about merge sort?

A Merge Sort works better than quick sort if data is accessed from slow sequential memory.

B Merge Sort is stable sort by nature

C Merge sort outperforms heap sort in most of the practical situations.

All of the above.

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 22 Explanation:
See Merge Sort and this.

Question 23

Wrong

Given an array where numbers are in range from 1 to n^6, which sorting algorithm can be used to sort these number in linear time?

Not possible to sort in linear time

Radix Sort

C Counting Sort

D Quick Sort

Sorting **QuickSort** **RadixSort** **CountingSort**
Discuss it

Question 23 Explanation:
See Radix Sort for explanation.

Question 24

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ (nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**
Discuss it

Question 24 Explanation:
Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 26

Correct

Let P be a QuickSort Program to sort numbers in ascending order using the first element as pivot. Let t1 and t2 be the number of comparisons made by P for the inputs {1, 2, 3, 4, 5} and {4, 1, 5, 3, 2} respectively. Which one of the following holds?

A t1 = 5

B t1 < t2

t1 > t2

D t1 = t2

Sorting **GATE-CS-2014-(Set-1)** **QuickSort**
Discuss it

Question 26 Explanation:
When first element or last element is chosen as pivot, Quick Sort's worst case occurs for the sorted arrays. In every step of quick sort, numbers are divided as per the following recurrence. T(n) = T(n-1) + O(n)

Question 27

Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

O(nLogn)

C Theta(nLogn)

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 27 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question
28

Question 28

Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 28 Explanation:

Insertion sort runs in Θ(n + f(n)) time, where f(n) denotes the number of inversion initially present in the array being sorted. Source: http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 29

Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 29 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 30

Wrong

Which of the following changes to typical QuickSort improves its performance on average and are generally done in practice.

```
1) Randomly picking up to make worst case less
   likely to occur.
2) Calling insertion sort for small sized arrays
   to reduce recursive calls.
3) QuickSort is tail recursive, so tail call
   optimizations can be done.
4) A linear time median searching algorithm is used
   to pick the median, so that the worst case time
   reduces to O(nLogn)
```

A 1 and 2

2, 3, and 4

1, 2 and 3

D 2, 3 and 4

Sorting GATE-CS-2015 (Mock Test) QuickSort
Discuss it

Question 30 Explanation:
The 4th optimization is generally not used, it reduces the worst case time complexity to O(nLogn), but the hidden constants are very high.

Question 31

Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting n(≥ 2) numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T (n/2) + cn$

$T(n) = T(n – 1) + T(0) + cn$

C $T(n) = 2T (n – 2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms Sorting GATE-CS-2015 (Set 1) QuickSort
Discuss it

Question 31 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 32

Wrong

Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

A 256

512

1024

D 2048

Sorting **GATE-CS-2015 (Set 3)** **MergeSort**
Discuss it

Question 32 Explanation:

```
Time complexity of merge sort is Θ(nLogn)

c*64Log64 is 30
c*64*6 is 30
c is 5/64

For time 6 minutes

5/64*nLogn = 6*60

nLogn = 72*64 = 512 * 9

n = 512.
```

Question 33

Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 1)**
Discuss it

Question 33 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and

recursive formula becomes T(n) = T(n-1) + Θ(n). An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 34

Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 34 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in Θ(n^2) time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in Θ(n) time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity $\Theta(n^2)$ [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 35

Wrong

Assume that we use Bubble Sort to sort n distinct elements in ascending order. When does the best case of Bubble Sort occur?

When elements are sorted in ascending order

B When elements are sorted in descending order

C When elements are not sorted by any order

There is no best case for Bubble Sort. It always takes O(n*n) time

Sorting BubbleSort
Discuss it

Question 36

Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A Θ(n)

B Θ(kn)

Θ(nlogn)

D $\Theta(n^2)$

Analysis of Algorithms Sorting RadixSort Gate IT 2008
Discuss it

Question 36 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 37

Correct

Consider an array of elements arr[5]= {5,4,3,2,1} , what are the steps of insertions done while doing insertion sort in the array.

4 5 3 2 1 3 4 5 2 1 2 3 4 5 1 1 2 3 4 5

B 5 4 3 1 2 5 4 1 2 3 5 1 2 3 4 1 2 3 4 5

C 4 3 2 1 5 3 2 1 5 4 2 1 5 4 3 1 5 4 3 2

D 4 5 3 2 1 2 3 4 5 1 3 4 5 2 1 1 2 3 4 5

Sorting **InsertionSort**
Discuss it

Question 37 Explanation:
In the insertion sort , just imagine that the first element is already sorted and all the right side Elements are unsorted, we need to insert all elements one by one from left to right in the sorted Array. Sorted : 5 unsorted : 4 3 2 1 Insert all elements less than 5 on the left (Considering 5 as the key) Now key value is 4 and array will look like this Sorted : 4 5 unsorted : 3 2 1 Similarly for all the cases the key will always be the newly inserted value and all the values will be compared to that key and inserted in to proper position.

Question 39

Wrong

Which of the following statements is correct with respect to insertion sort ?

```
*Online - can sort a list at runtime
*Stable - doesn't change the relative
          order of elements with equal keys.
```

Insertion sort is stable, online but not suited well for large number of elements.

Insertion sort is unstable and online

C Insertion sort is online and can be applied to more than 100 elements

D Insertion sort is stable & online and can be applied to more than 100 elements

Sorting **InsertionSort**
Discuss it

Question 39 Explanation:
Time taken by algorithm is good for small number of elements, but increases quadratically for large number of elements.

Question 40

Wrong

Consider the array A[]= {6,4,8,1,3} apply the insertion sort to sort the array . Consider the cost associated with each sort is 25 rupees , what is the total cost of the insertion sort when element 1 reaches the first position of the array ?

50

B 25

75

D 100

Sorting **C Arrays** **InsertionSort**
Discuss it

Question 40 Explanation:
When the element 1 reaches the first position of the array two comparisons are only required hence 25 * 2= 50 rupees. *step 1: 4 6 8 1 3 . *step 2: 1 4 6 8 3.

Question 42

Wrong

What is the best sorting algorithm to use for the elements in array are more than 1 million in general?

Merge sort.

B Bubble sort.

Quick sort.

D Insertion sort.

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 42 Explanation:
Most practical implementations of Quick Sort use randomized version. The randomized version has expected time complexity of O(nLogn). The worst case is possible in randomized version also, but worst case doesn't occur for a particular pattern (like sorted array) and randomized Quick Sort works well in practice. Quick Sort is also a cache friendly sorting algorithm as it has good locality of

reference when used for arrays. Quick Sort is also tail recursive, therefore tail call optimizations is done.

Question 43

Wrong

Which of the below given sorting techniques has highest best-case runtime complexity.

A Quick sort

Selection sort

C Insertion sort

Bubble sort

Sorting **GATE 2017 Mock**
Discuss it

Question 43 Explanation:
Quick sort best case time complexity is O(n logn)
Selection sort best case time complexity is O(n^2)
Insertion sort best case time complexity is O(n)
Bubble sort best case time complexity is O(n)

Question 45

Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 45 Explanation:

In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 46

Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

```
(i) 1, 2, 3,......., n
(ii) n, n-1, n-2,......, 2, 1
```

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 46 Explanation:
Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 50

Wrong

You are given a sequence of n elements to sort. The input sequence consists of n/k subsequences,each containing k elements. The elements in a given subsequence are all smaller than the elements in the succeeding subsequence and larger than the elements in the preceding subsequence. Thus, all that is needed to sort the whole sequence of length n is to sort the k elements in each of the n/k subsequences. The lower bound on the number of comparisons needed to solve this variant of the sorting problem is

A Ω (n)

Ω (n/k)

Ω (nlogk)

D Ω (n/klogn/k)

Sorting UGC-NET CS 2017 Nov - III
Discuss it

C E BINE SUS

Question 51

Wrong

Which of the following is true for computation time in insertion, deletion and finding maximum and minimum element in a sorted array ?

A Insertion – 0(1), Deletion – 0(1), Maximum – 0(1), Minimum – 0(l)

B Insertion – 0(1), Deletion – 0(1), Maximum – 0(n), Minimum – 0(n)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(1), Minimum – 0(1)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(n), Minimum – 0(n)

Sorting UGC NET CS 2017 Jan - II
Discuss it

Question 51 Explanation:

1. In a sorted array, if we want to insert or delete then we have to traverse whole array and check where is the suitable position, so it will take O(n).
2. If array is sorted then end position will tell the maximum or minimum, so finding maximum or minimum will take O(1).

So, option (C) is correct.

Question 52

Correct

Consider the following sorting algorithms. I. Quicksort II. Heapsort III. Mergesort Which of them perform in least time in the worst case?

A I and II only

II and III only

C III only

D I, II and III

Sorting **ISRO CS 2014**
Discuss it

Question 52 Explanation:
Worst case complexity of Heap Sort and Merge Sort is O(n log n) whereas the worst case complexity of Quicksort is $O(n^2)$. (B) option is correct.

Question 53

Wrong

Which of the following algorithms sort n integers, having the range 0 to $(n^2 - 1)$, in ascending order in O(n) time ?

A Selection sort

B Bubble sort

Radix sort

Insertion sort

Sorting **UGC NET CS 2015 Jun - II**
Discuss it

Question 53 Explanation:

- Selection sort takes $O(n^2)$ time.
- Bubble sort takes $O(n^2)$ time.
- Radix sort takes O(n) time.
- Insertion sort takes $O(n^2)$ time.

So, option (C) is correct.

Question 54

Correct

Which one of the following in place sorting algorithms needs the minimum number of swaps?

A Quick sort

B Insertion sort

Selection sort

D Heap sort

Sorting **ISRO CS 2011**
Discuss it

Question 54 Explanation:
Selection sort takes minimum number of swaps to sort an array. It takes maximum of O(n) comparisons to sort an array with n elements. Refer: Selection sort

Question 56

Correct

Selection sort algorithm design technique is an example of

Greedy method

B Divide-and-conquer

C Dynamic Programming

D Backtracking

Sorting **ISRO CS 2007**
Discuss it

Question 56 Explanation:
The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array. 1) The subarray which is already sorted. 2) Remaining subarray which is unsorted. In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray. Clearly, it is a greedy approach to sort the array. Option (A) is correct.

Question 58

Wrong

You have to sort a list L, consisting of a sorted list followed by a few ‘random’ elements. Which of the following sorting method would be most suitable for such a task?

A Bubble sort

B Selection sort

Quick sort

Insertion sort

Sorting **UGC NET CS 2014 Dec - II**
Discuss it

Question 58 Explanation:
For a sorted list with few random numbers:

- Bubble sort will take $O(n^2)$ time in best case.
- Selection sort will take $O(n^2)$ time in best case.
- Quick sort will take $O(n^2)$ time in this case because it is the worst case for quick sort.
- Insertion sort will take O(n) So, option (D) is correct.

Question 1

Correct

Which of the following algorithms is NOT a divide & conquer algorithm by nature?

A Euclidean algorithm to compute the greatest common divisor

Heap Sort

C Cooley-Tukey fast Fourier transform

D Quick Sort

Divide and Conquer
Discuss it

Question 1 Explanation:
See Divide and Conquer

Question
2

Question 3

Wrong

Consider the polynomial p(x) = a0 + a1x + a2x^2 +a3x^3, where ai != 0, for all i. The minimum number of multiplications needed to evaluate p on an input x is:

3

B 4

6

D 9

Divide and Conquer
Discuss it

Question 3 Explanation:
Multiplications can be minimized using following order for evaluation of the given expression. p(x) = a0 + x(a1 + x(a2 + a3x))

Question 4

Correct

Maximum Subarray Sum problem is to find the subarray with maximum sum. For example, given an array {12, -13, -5, 25, -20, 30, 10}, the maximum subarray sum is 45. The naive solution for this problem is to calculate sum of all subarrays starting with every element and return the maximum of all. We can solve this using Divide and Conquer, what will be the worst case time complexity using Divide and Conquer.

A O(n)

O(nLogn)

C O(Logn)

D O(n^2)

Divide and Conquer
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/divide-and-conquer-maximum-sum-subarray/

Question 5

Correct

Consider a situation where you don't have function to calculate power (pow() function in C) and you need to calculate x^n where x can be any number and n is a positive integer. What can be the best possible time complexity of your power function?

A O(n)

B O(nLogn)

C O(LogLogn)

O(Logn)

Divide and Conquer
Discuss it

Question 5 Explanation:
We can calculate power using divide and conquer in O(Logn) time. See http://www.geeksforgeeks.org/write-a-c-program-to-calculate-powxn/.

Question 6

Wrong

Consider the problem of searching an element x in an array 'arr[]' of size n. The problem can be solved in O(Logn) time if. 1) Array is sorted 2) Array is sorted and rotated by k. k is given to you and k <= n 3) Array is sorted and rotated by k. k is NOT given to you and k <= n 4) Array is not sorted

A 1 Only

1 & 2 only

1, 2 and 3 only

D 1, 2, 3 and 4

Divide and Conquer
Discuss it

Question 6 Explanation:
See http://www.geeksforgeeks.org/search-an-element-in-a-sorted-and-pivoted-array/

Question 9

Wrong

Consider the problem of computing min-max in an unsorted array where min and max are minimum and maximum elements of array. Algorithm A1 can compute min-max in a1 comparisons without divide and conquer. Algorithm A2 can compute min-max in a2 comparisons by scanning the array linearly. What could be the relation between a1 and a2 considering the worst case scenarios?

A a1 < a2

a1 > a2

a1 = a2

D Depends on the input

Divide and Conquer **GATE 2017 Mock**
Discuss it

Question 9 Explanation:
When Divide and Conquer is used to find the minimum-maximum element in an array, Recurrence relation for the number of comparisons is
T(n) = 2T(n/2) + 2 where 2 is for comparing the minimums as well the maximums of the left and right subarrays
On solving, T(n) = 1.5n - 2.
While doing linear scan, it would take 2*(n-1) comparisons in the worst case to find both minimum as well maximum in one pass.

Question 1

Correct

Which of the following standard algorithms is not a Greedy algorithm?

A Dijkstra's shortest path algorithm

B Prim's algorithm

C Kruskal algorithm

D Huffman Coding

Bellmen Ford Shortest path algorithm

Greedy Algorithms
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/ and http://www.geeksforgeeks.org/dynamic-programming-set-23-bellman-ford-algorithm/

Question 3

Wrong

A networking company uses a compression technique to encode the message before transmitting over the network. Suppose the message contains the following characters with their frequency:

character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Note : Each character in input message takes 1 byte. If the compression technique used is Huffman Coding, how many bits will be saved in the message?

A 224

800

576

D 324

Greedy Algorithms
Discuss it

Question 3 Explanation:

```
Total number of characters in the message = 100.
Each character takes 1 byte. So total number of bits needed = 800.

After Huffman Coding, the characters can be represented with:
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
Total number of bits needed = 224
Hence, number of bits saved = 800 - 224 = 576
See here for complete explanation and algorithm.
```

Question 4

Correct

What is the time complexity of Huffman Coding?

A O(N)

O(NlogN)

C O(N(logN)^2)

D O(N^2)

Greedy Algorithms
Discuss it

Question 4 Explanation:
O(nlogn) where n is the number of unique characters. If there are n nodes, extractMin() is called 2*(n – 1) times. extractMin() takes O(logn) time as it calles minHeapify(). So, overall complexity is O(nlogn). See here for more details of the algorithm.

Question 5

Wrong

In question #2, which of the following represents the word *"dead"*?

A 1011111100101

B 0100000011010

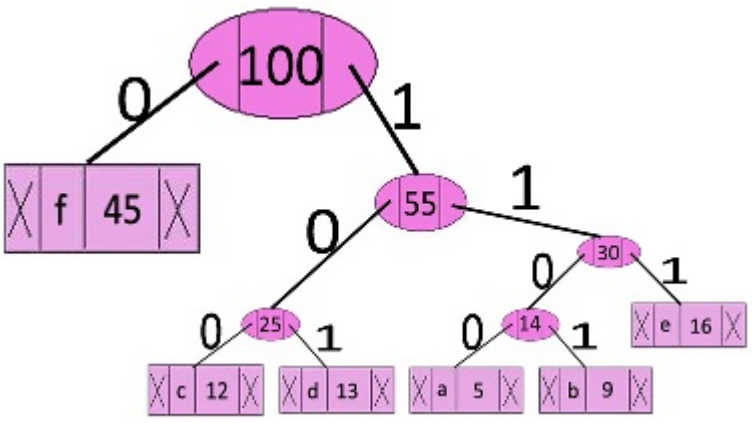
Both A and B

None of these

Greedy Algorithms
Discuss it

Question 5 Explanation:

The *Huffman Tree* generated is:



```
character   code-word
    f          0
    c          100
    d          101
    a          1100
    b          1101
    e          111
```

The word *dead* can be represented as: 101 111 1100 101 However, the alternative codeword can also be found by assigning 1 to the left edge and 0 to the right edge of the tree, i.e. *dead* can also be represented as: 010 000 0011 010 See here for more details of the algorithm.

Question 6

Wrong

Which of the following is true about Kruskal and Prim MST algorithms? Assume that Prim is implemented for adjacency list representation using Binary Heap and Kruskal is implemented using union by rank.

Worst case time complexity of both algorithms is same.

B Worst case time complexity of Kruskal is better than Prim

Worst case time complexity of Prim is better than Kruskal

Greedy Algorithms
Discuss it

Question 6 Explanation:
See following Kruskal's Minimum Spanning Tree Algorithm Prim's MST for Adjacency List Representation

Question 7

Wrong

Which of the following is true about Huffman Coding.

A Huffman coding may become lossy in some cases

B Huffman Codes may not be optimal lossless codes in some cases

In Huffman coding, no code is prefix of any other code.

All of the above

Greedy Algorithms
Discuss it

Question 7 Explanation:
Huffman coding is a lossless data compression algorithm. The codes assigned to input characters are Prefix Codes, means the codes are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding.

Question 8

Correct

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. Which of the following is the Huffman code for the letter a, b, c, d, e, f?

0, 10, 110, 1110, 11110, 11111

B 11, 10, 011, 010, 001, 000

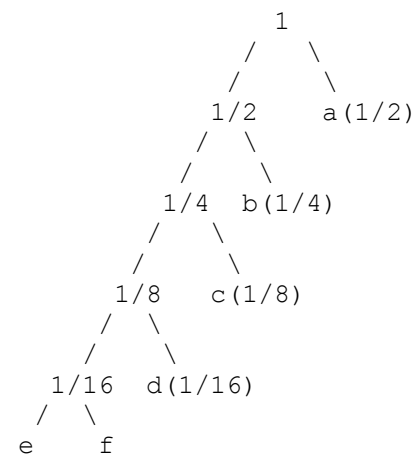
C 11, 10, 01, 001, 0001, 0000

D 110, 100, 010, 000, 001, 111

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 8 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.
```



Question

9

Question 9

Wrong

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. What is the average length of Huffman codes?

3

B 2.1875

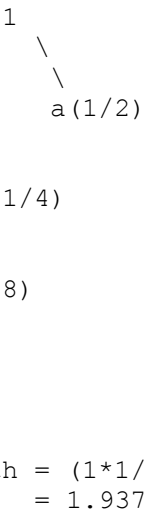
C 2.25

1.9375

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 9 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.
```




```
The average length = (1*1/2 + 2*1/4 + 3*1/8 + 4*1/16 + 5*1/32 + 5*1/32)
                   = 1.9375
```

Question 11

Wrong

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	Weight (in Kgs)	Value (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}. The value of $V_{opt} - V_{greedy}$ is _____ .

Note -This was Numerical Type question.

16

8

C 44

D 60

Greedy Algorithms Dynamic Programming GATE CS 2018

Discuss it

Question 11 Explanation:

Item No	Weight	Value	Value/Weight
1	10	60	6
2	7	28	4
3	4	20	5
4	2	24	12

After sorting :

Item No	Weight	Value	Value/Weight
4	2	24	12
1	10	60	6
3	4	20	5
2	7	28	4

First we will pick item_4 (Value weight ratio is highest). Second highest is item_1, but cannot be picked because of its weight. Now item_3 shall be picked. item_2 cannot be included because of its weight. Therefore, overall profit by V_{greedy} = 20+24 = 44 Hence, $\mathbf{V_{opt} - V_{greedy} = 60\text{-}44 = 16}$ So, answer is 16.

Question 12

Wrong

A text is made up of the characters a, b, c, d, e each occurring with the probability 0.11, 0.40, 0.16, 0.09 and 0.24 respectively. The optimal Huffman coding technique will have the average length of:

A 2.40

2.16

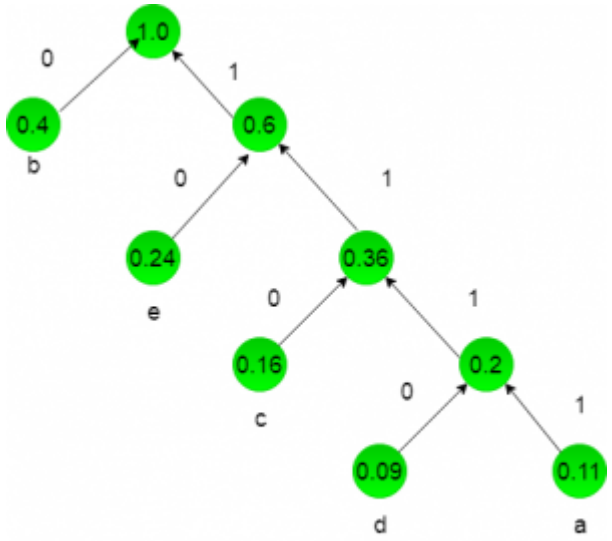
2.26

D 2.15

Greedy Algorithms UGC-NET CS 2017 Nov - III
Discuss it

Question 12 Explanation:

a = 0.11 b = 0.40 c = 0.16 d = 0.09 e = 0.24 we will draw a huffman tree:



Text now huffman coding for character:

```
 a = 1111
      b = 0
      c = 110
      d = 1111
      e = 10
lenghth for each character = no of bits * frequency of occurence:
a = 4 * 0.11
  = 0.44
b = 1 * 0.4
  =  0.4
c = 3 * 0.16
  = 0.48
d = 4 * 0.09
  =  0.36
e = 2 * 0.24
  = 0.48
Now add these lenght for average length:
 0.44 + 0.4 + 0.48 + 0.36 + 0.48 = 2.16
```

So, option (B) is correct.

Question 13

Correct

Six files F1, F2, F3, F4, F5 and F6 have 100, 200, 50, 80, 120, 150 records respectively. In what order should they be stored so as to optimize act. Assume each file is accessed with the same frequency

F3, F4, F1, F5, F6, F2

B F2, F6, F5, F1, F4, F3

C F1, F2, F3, F4, F5, F6

D Ordering is immaterial as all files are accessed with the same frequency.

Greedy Algorithms **ISRO CS 2015**
Discuss it

Question 13 Explanation:

```
This question is based on the Optimal Storage on Tape problem
which uses greedy approach to find the optimal time to retrieve them.
There are n programs of length L that are to be stored on a computer
tape. Associated with each program i is a length Li. So in order to
retrieve these programs most optimally, we need to store them in the
non-decreasing order of length Li.
So, the correct order is F3, F4, F1, F5, F6, F2
```

Question 14

Correct

Consider a job scheduling problem with 4 jobs J_1, J_2, J_3, J_4 and with corresponding deadlines: (d_1, d_2, d_3, d_4) = (4, 2, 4, 2). Which of the following is not a feasible schedule without violating any job schedule?

A J_2, J_4, J_1, J_3

J_4, J_1, J_2, J_3

C J_4, J_2, J_1, J_3

D J_4, J_2, J_3, J_1

Greedy Algorithms **ISRO CS 2007**
Discuss it

Question
15

Question 15

Correct

What is the other name of Dijkstra algorithm?

single-source shortest path problem

B multiple-source shortest path problem

C multiple-destination shortest path problem

D single-destination shortest path problem

Greedy Algorithms
Discuss it

Question 2

Correct

We use dynamic programming approach when

A We need an optimal solution

The solution has optimal substructure

C The given problem can be reduced to the 3-SAT problem

D It's faster than Greedy

Dynamic Programming
Discuss it

Question 2 Explanation:
http://www.geeksforgeeks.org/dynamic-programming-set-2-optimal-substructure-property/ Option (D) is incorrect because Greedy algorithms are generally faster than Dynamic programming. See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/

Question 3

Wrong

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array A[0 :n-1] is given below. Let Li denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$

For all i such that $0 \le i \le n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if } A[i] < A[i+1] \\ 1 & \text{Otherwise} \end{cases}$$

Finally the length of the longest monotonically increasing sequence is $\text{Max}(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is TRUE?

The algorithm uses dynamic programming paradigm

The algorithm has a linear complexity and uses branch and bound paradigm

C The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm

D The algorithm uses divide and conquer paradigm.

Dynamic Programming
Discuss it

Question 3 Explanation:
Please see http://www.geeksforgeeks.org/dynamic-programming-set-3-longest-increasing-subsequence/

Question 4

Wrong

Kadane algorithm is used to find:

Maximum sum subsequence in an array

Maximum sum subarray in an array

C Maximum product subsequence in an array

D Maximum product subarray in an array

Dynamic Programming
Discuss it

Question 4 Explanation:
Kadane algorithm is used to find the maximum sum subarray in an array. It runs in O(n) time complexity. See details of the algorithm here.

Question 5

Wrong

Four matrices M1, M2, M3 and M4 of dimensions pxq, qxr, rxs and sxt respectively can be multiplied is several ways with different number of total scalar multiplications. For example, when multiplied as ((M1 X M2) X (M3 X M4)), the total number of multiplications is pqr + rst + prt. When multiplied as (((M1 X M2) X M3) X M4), the total number of scalar multiplications is pqr + prs + pst. If p = 10, q = 100, r = 20, s = 5 and t = 80, then the number of scalar multiplications needed is

248000

B 44000

19000

D 25000

Dynamic Programming
Discuss it

Question 5 Explanation:
It is basically matrix chain multiplication problem. We get minimum number of multiplications using ((M1 X (M2 X M3)) X M4). Total number of multiplications = 100x20x5 (for M2 x M3) + 10x100x5 + 10x5x80 = 19000.

Question 6

Wrong

The subset-sum problem is defined as follows. Given a set of n positive integers, S = {a1 ,a2 ,a3 ,…,an} and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array X, with n rows and W+1 columns. X[i, j],1 <= i <= n, 0 <= j <= W, is TRUE if and only if there is a subset of {a1 ,a2 ,...,ai} whose elements sum to j. Which of the following is valid for 2 <= i <= n and ai <= j <= W?

A X[i, j] = X[i - 1, j] ∨ X[i, j -ai]

X[i, j] = X[i - 1, j] ∨ X[i - 1, j - ai]

C X[i, j] = X[i - 1, j] ∧ X[i, j - ai]

X[i, j] = X[i - 1, j] ∧ X[i -1, j - ai]

Dynamic Programming
Discuss it

Question 6 Explanation:
X[I, j] (2 <= i <= n and ai <= j <= W), is true if any of the following is true 1) Sum of weights excluding ai is equal to j, i.e., if X[i-1, j] is true. 2) Sum of weights including ai is equal to j, i.e., if X[i-1, j-ai] is true so that we get (j – ai) + ai as j See http://www.geeksforgeeks.org/dynamic-programming-subset-sum-problem/ for details.

Question 7

Wrong

In the above question, which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

X[1, W]

B X[n ,0]

X[n, W]

D X[n -1, n]

Dynamic Programming
Discuss it

Question 7 Explanation:
If we get the entry X[n, W] as true then there is a subset of {a1, a2, .. an} that has sum as W.
Reference: http://en.wikipedia.org/wiki/Subset_sum_problem

Question 8

Wrong

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences X[m] and Y[n] of lengths m and n respectively, with indexes of X and Y starting from 0. We wish to find the length of the longest common sub-sequence(LCS) of X[m] and Y[n] as l(m,n), where an incomplete recursive definition for the function l(i,j) to compute the length of The LCS of X[m] and Y[n] is given below:

```
l(i,j) = 0, if either i=0 or j=0
       = expr1, if i,j > 0 and X[i-1] = Y[j-1]
       = expr2, if i,j > 0 and X[i-1] != Y[j-1]
```

expr1 ≡ l(i-1, j) + 1

B expr1 ≡ l(i, j-1)

expr2 ≡ max(l(i-1, j), l(i, j-1))

D expr2 ≡ max(l(i-1,j-1),l(i,j))

Dynamic Programming **GATE-CS-2009**
Discuss it

Question 8 Explanation:
In Longest common subsequence problem, there are two cases for X[0..i] and Y[0..j]

```
1) The last characters of two strings match.
   The length of lcs is length of lcs of X[0..i-1] and Y[0..j-1]
2) The last characters don't match.
   The length of lcs is max of following two lcs values
   a) LCS of X[0..i-1] and Y[0..j]
   b) LCS of X[0..i] and Y[0..j-1]
```

Question 9

Wrong

Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B. Then x + 10y = ___.

A 33

B 23

43

34

Dynamic Programming GATE-CS-2014-(Set-2)
Discuss it

Question 9 Explanation:
//The LCS is of length 4. There are 3 LCS of length 4 "qprr", "pqrr" and qpqr A subsequence is a sequence that can be derived from another sequence by selecting zero or more elements from it, without changing the order of the remaining elements. Subsequence need not be contiguous. Since the length of given strings A = “qpqrr” and B = “pqprqrp” are very small, we don’t need to build a 5x7 matrix and solve it using dynamic programming. Rather we can solve it manually just by brute force. We will first check whether there exist a subsequence of length 5 since min_length(A,B) = 5. Since there is no subsequence , we will now check for length 4. “qprr”, “pqrr” and “qpqr” are common in both strings. X = 4 and Y = 3 X + 10Y = 34 This solution is contributed by **Pranjul Ahuja**

Question 10

Correct

Let A1, A2, A3, and A4 be four matrices of dimensions 10 x 5, 5 x 20, 20 x 10, and 10 x 5, respectively. The minimum number of scalar multiplications required to find the product A1A2A3A4 using the basic matrix multiplication method is

1500

B 2000

C 500

D 100

Dynamic Programming GATE-CS-2016 (Set 2)
Discuss it

Question 10 Explanation:
We have many ways to do matrix chain multiplication because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result of the matrix chain multiplication obtained will remain the same. Here we have four matrices A1, A2, A3, and A4, we would have: ((A1A2)A3)A4 = ((A1(A2A3))A4) = (A1A2)(A3A4) = A1((A2A3)A4) = A1(A2(A3A4)). However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency. Here, A1 is a 10 × 5 matrix, A2 is a 5 x 20 matrix, and A3 is a 20 x 10 matrix, and A4 is 10 x 5. If we multiply two matrices A and B of order l x m and m x n respectively,then the number of scalar multiplications in the multiplication of A and B will be lxmxn. Then, The number of scalar multiplications required in the following sequence of matrices will be : A1((A2A3)A4) = (5 x 20 x 10) + (5 x 10 x 5) + (10 x 5 x 5) = 1000 + 250 + 250 = 1500. All other parenthesized options will require number of multiplications more than 1500.

Question 12

Wrong

Consider the following two sequences :

X = < B, C, D, C, A, B, C >, and
Y = < C, A, D, B, C, B >
The length of longest common subsequence of X and Y is :

A 5

3

4

D 2

Dynamic Programming UGC-NET CS 2017 Nov - III
Discuss it

Question 13

Correct

Consider a sequence F_{00} defined as : $F_{00}(0) = 1$, $F_{00}(1) = 1$ $F_{00}(n) = 10 * F_{00}(n - 1) + 100$ $F_{00}(n - 2)$ for $n \geq 2$ Then what shall be the set of values of the sequence F_{00} ?

(1, 110, 1200)

B (1, 110, 600, 1200)

C (1, 2, 55, 110, 600, 1200)

D (1, 55, 110, 600, 1200)

Dynamic Programming **UGC NET CS 2017 Jan - II**
Discuss it

Question 13 Explanation:

```
F00(0) = 1, F00(1) = 1
F00(n) = 10 * F00(n - 1) + 100
F00(2) = 10 * F00(1) + 100
= 10 * 1 + 100
= 10 + 100
= 110
Similarly:
F00(3) = 10 * F00(2) + 100
= 10 * 110 + 100
= 1100 + 100
= 1200
The sequence will be (1, 110, 1200).
```

So, (A) will be the answer.

Question 14

Correct

The following paradigm can be used to find the solution of the problem in minimum time: Given a set of non-negative integer, and a value K, determine if there is a subset of the given set with sum equal to K:

A Divide and Conquer

Dynamic Programming

C Greedy Algorithm

D Branch and Bound

Dynamic Programming **ISRO CS 2018**
Discuss it

Question 14 Explanation:
Given problem is Subset-sum problem in which a set of non-negative integers, and a value sum is given, to determine if there is a subset of the given set with sum equal to given sum. With recursion technique, time complexity of the above problem is exponential. We can solve the problem in Pseudo-polynomial time using Dynamic programming. Refer: Subset Sum Problem Option (B) is correct

Question 15

Wrong

What happens when a top-down approach of dynamic programming is applied to any problem?

It increases both, the time complexity and the space complexity

It increases the space complexity and decreases the time complexity.

C It increases the time complexity and decreases the space complexity

D It decreases both, the time complexity and the space complexity

Dynamic Programming
Discuss it

Question 1

Correct

Which of the following algorithms can be used to most efficiently determine the presence of a cycle in a given graph ?

Depth First Search

B Breadth First Search

C Prim's Minimum Spanning Tree Algorithm

D Kruskal' Minimum Spanning Tree Algorithm

Graph Traversals
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/applications-of-depth-first-search/ and
http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 2

Correct

Traversal of a graph is different from tree because

There can be a loop in graph so we must maintain a visited flag for every vertex

B DFS of a graph uses stack, but inorrder traversal of a tree is recursive

C BFS of a graph uses queue, but a time efficient BFS of a tree is recursive.

D All of the above

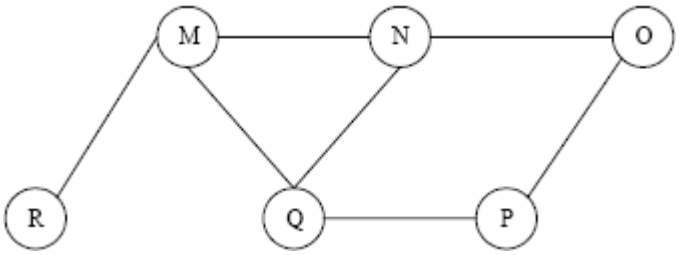
Graph Traversals
Discuss it

Question 2 Explanation:
See http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 4

Correct

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is



A MNOPQR

B NQMPOR

QMNPRO

D QMNPOR

Graph Traversals
Discuss it

Question 4 Explanation:
Option (A) is MNOPQR. It cannot be a BFS as the traversal starts with M, but O is visited before N and Q. In BFS all adjacent must be visited before adjacent of adjacent. Option (B) is NQMPOR. It also cannot be BFS, because here, P is visited before O. (C) and (D) match up to QMNP. We see that M was added to the queue before N and P (because M comes before NP in QMNP). Because R is M's neighbor, it gets added to the queue before the neighbor of N and P (which is O). Thus, R is visited before O.

Question 5

Wrong

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE CS 2000)

A {u,v} must be an edge in G, and u is a descendant of v in T

B {u,v} must be an edge in G, and v is a descendant of u in T

If {u,v} is not an edge in G then u is a leaf in T

If {u,v} is not an edge in G then u and v must have the same parent in T

Graph Traversals
Discuss it

Question 5 Explanation:

```
In DFS, if 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
      u
    /   \
   v     w
  /     / \
 x     y   z

2) 'u' is a leaf node.
      w
    /   \
   x     v
  /     / \
 u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 7

Correct

Make is a utility that automatically builds executable programs and libraries from source code by reading files called makefiles which specify how to derive the target program. Which of the following standard graph algorithms is used by Make.

A Strongly Connected Components

Topological Sorting

C Breadth First Search

D Dijkstra's Shortest Path

Graph Traversals
Discuss it

Question 7 Explanation:
Make can decide the order of building a software using topological sorting. Topological sorting produces the order considering all dependencies provide by makefile. See following for details.
Topological Sorting

Question 9

Correct

Which of the following condition is sufficient to detect cycle in a directed graph?

A There is an edge from currently being visited node to an already visited node.

There is an edge from currently being visited node to an ancestor of currently visited node in DFS forest.

C Every node is seen twice in DFS.

D None of the bove

Graph Traversals
Discuss it

Question 9 Explanation:
See Detect Cycle in a Graph

Question 10

Wrong

Is following statement true/false If a DFS of a directed graph contains a back edge, any other DFS of the same graph will also contain at least one back edge. Source:
http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-s2009-sol.pdf

True

False

Graph Traversals
Discuss it

Question 10 Explanation:
A back edge means a cycle in graph. So if there is a cycle, all DFS traversals would contain at least one back edge.

Question 14

Wrong

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G? Assume that the graph is represented using adjacency matrix.

A O(n)

O(m+n)

O(n^2)

D O(mn)

Graph Traversals **GATE-CS-2014-(Set-1)**
Discuss it

Question 14 Explanation:
Depth First Search of a graph takes O(m+n) time when the graph is represented using adjacency list. In adjacency matrix representation, graph is represented as an "n x n" matrix. To do DFS, for every vertex, we traverse the row corresponding to that vertex to find all adjacent vertices (In adjacency list representation we traverse only the adjacent vertices of the vertex). Therefore time complexity becomes O(n^2)

Question 15

Correct

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing.

A the shortest path between every pair of vertices.

the shortest path from W to every vertex in the graph.

C the shortest paths from W to only those nodes that are leaves of T.

D the longest path in the graph

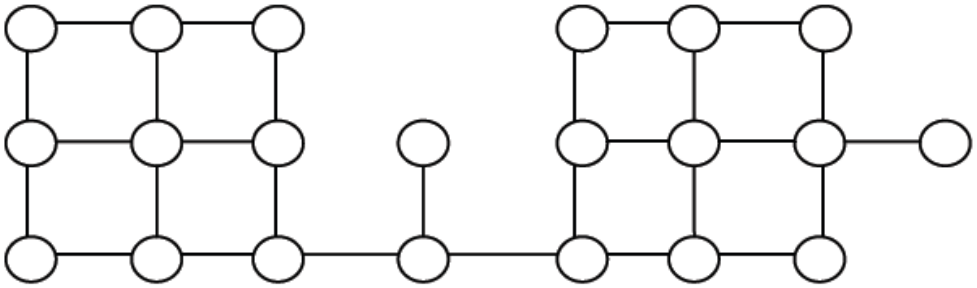
Graph Traversals **GATE-CS-2014-(Set-2)**
Discuss it

Question 15 Explanation:
BFS always produces shortest path from source to all other vertices in an unweighted graph.

Question 16

Wrong

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is



_____.

A 17

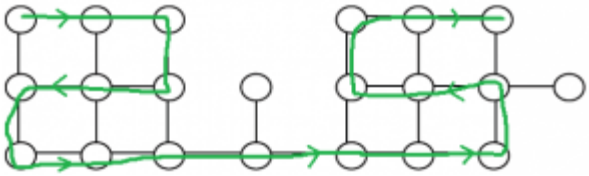
B 18

19

20

Graph Traversals **GATE-CS-2014-(Set-3)**
Discuss it

Question 16 Explanation:
The following diagram shows the worst case situation where the recursion tree has maximum depth.



So the recursion depth is 19 (including the first node).

Question 17

Wrong

Let T be a depth first search tree in an undirected graph G. Vertices u and n are leaves of this tree T. The degrees of both u and n in G are at least 2. which one of the following statements is true?

There must exist a vertex w adjacent to both u and n in G

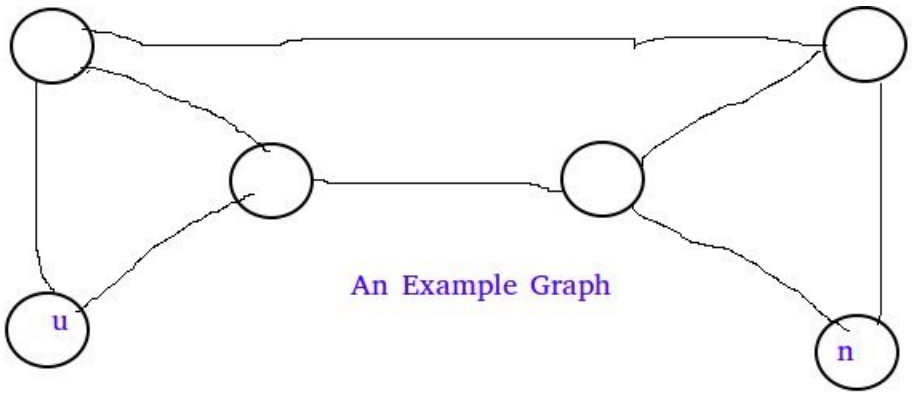
B There must exist a vertex w whose removal disconnects u and n in G

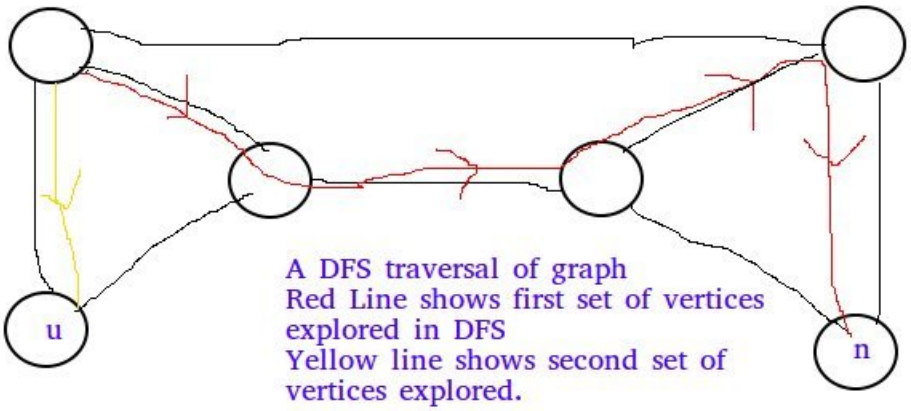
C There must exist a cycle in G containing u and n

There must exist a cycle in G containing u and all its neighbours in G.

Graph Traversals **GATE-CS-2006**
Discuss it

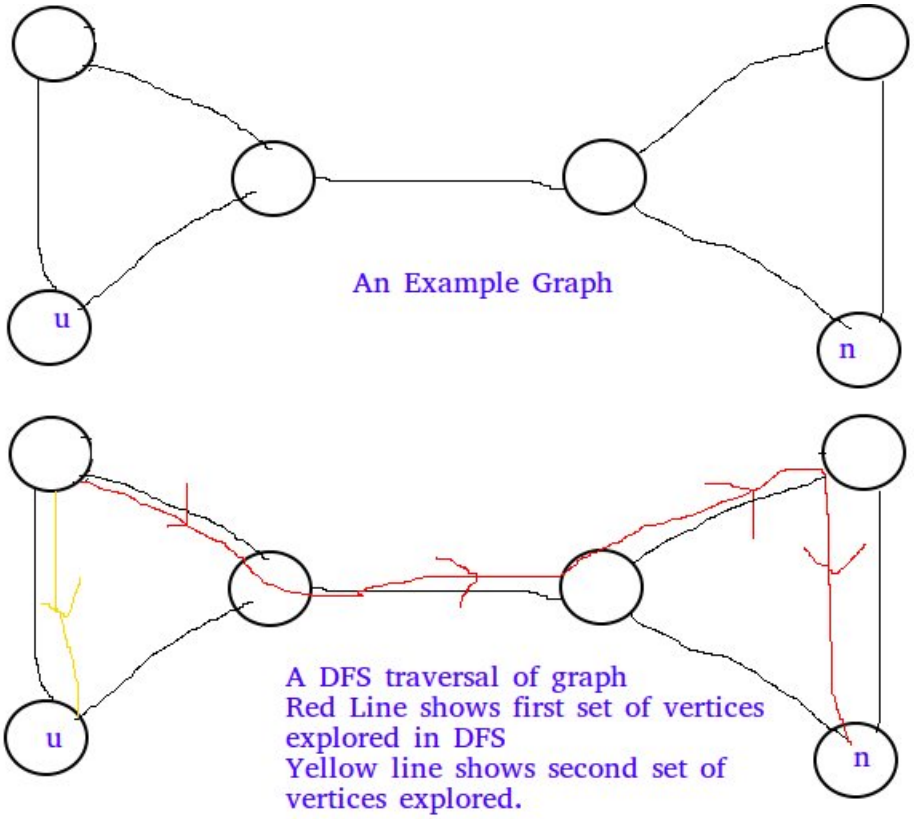
Question 17 Explanation:
Below example shows that A and B are FALSE:





Below

example shows C is false:



Question
18

Question 19

Correct

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

A k

B k + 1

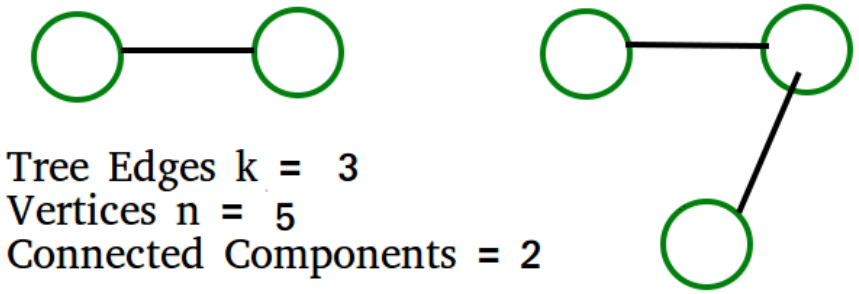
C n - k - 1

n - k

Graph Traversals **Gate IT 2005**
Discuss it

Question 19 Explanation:

Tree edges are the edges that are part of DFS tree. If there are x tree edges in a tree, then x+1 vertices in the tree. The output of DFS is a forest if the graph is disconnected. Let us see below simple example where graph is disconnected.

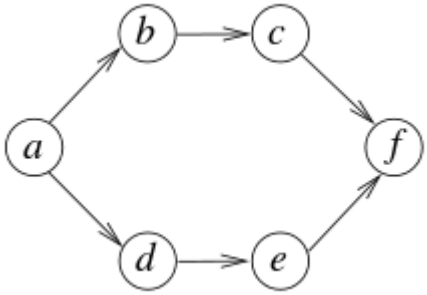


The above example matches with D option **More Examples:** 1) All vertices of Graph are connected. k must be n-1. We get number of connected components = n- k = n - (n-1) = 1 2) No vertex is connected. k must be 0. We get number of connected components = n- k = n - 0 = n

Question
20

Question 20

Wrong



Consider the following directed graph. The number of different topological orderings of the vertices of the graph is Note : This question was asked as Numerical Answer Type.

A 1

B 2

4

6

Graph Traversals **GATE-CS-2016 (Set 1)**
Discuss it

Question 20 Explanation:

Following are different 6 Topological Sortings

```
a-b-c-d-e-f
a-d-e-b-c-f
a-b-d-c-e-f
a-d-b-c-e-f
a-b-d-e-c-f
a-d-b-e-c-f
```

Question 21

Correct

In an adjacency list representation of an undirected simple graph G = (V, E), each edge (u, v) has two adjacency list entries: [v] in the adjacency list of u, and [u] in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If |E| = m and |V | = n, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A $\Theta(n^2)$

$\Theta(m+n)$

C $\Theta(m^2)$

D $\Theta(n^4)$

Graph Traversals **GATE-CS-2016 (Set 2)**
Discuss it

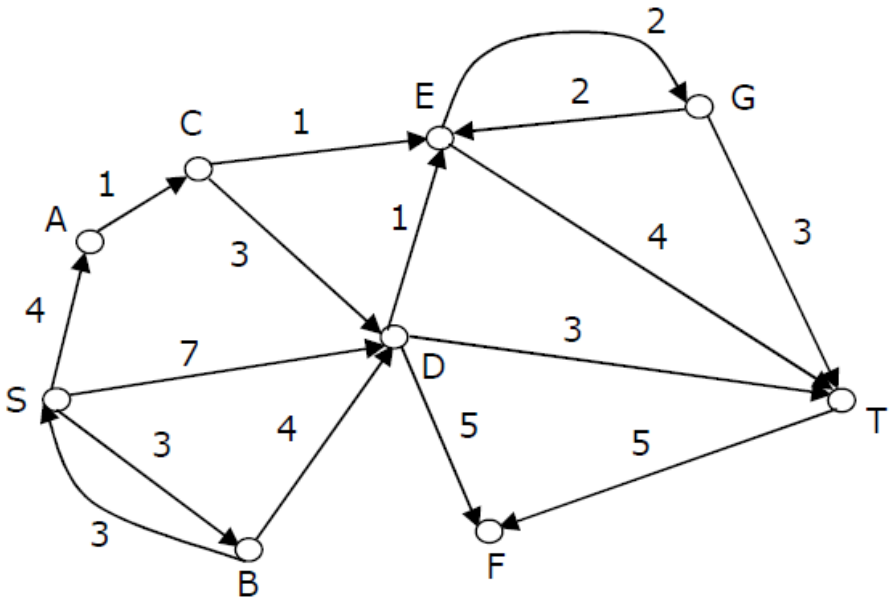
Question 21 Explanation:
First you need to find twins of each node. You can do this using **level order traversal (i.e., BFS)** once. Time complexity of BFS is Θ(m +n). And you have to use linked list for representation which is extra space (but memory size is not a constraint here). Final, time complexity is Θ(m + n) to set twin pointer. Option (B) is correct.

Question 1

Wrong

Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijstra?s shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is

discovered.



A SDT

SBDT

C SACDT

SACET

Graph Shortest Paths
Discuss it

Question 1 Explanation:
See Dijkstra's shortest path algorithm When the algorithm reaches vertex 'C', the distance values of 'D' and 'E' would be 7 and 6 respectively. So the next picked vertex would be 'E'

Question 2

Wrong

To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

Queue

B Stack

Heap

D B-Tree

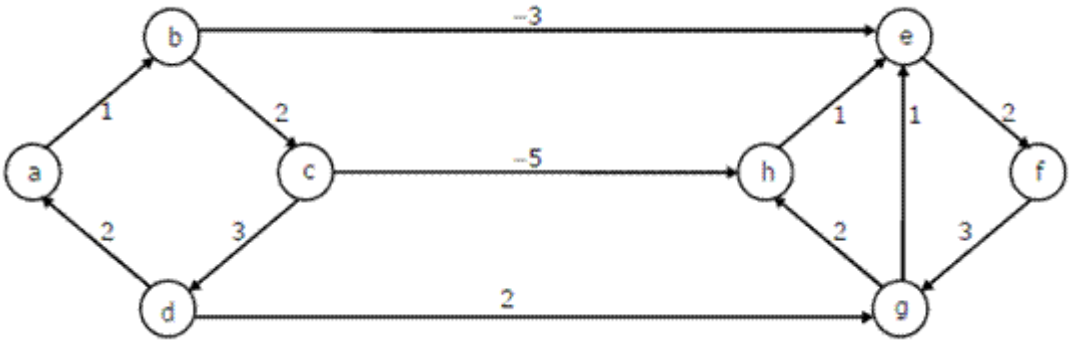
Graph Shortest Paths
Discuss it

Question 2 Explanation:
The shortest path in an un-weighted graph means the smallest number of edges that must be traversed in order to reach the destination in the graph. This is the same problem as solving the weighted version where all the weights happen to be 1. If we use Queue (FIFO) instead of Priority Queue (Min Heap), we get the shortest path in linear time O(|V| + |E|). Basically we do BFS traversal of the graph to get the shortest paths.

Question 3

Wrong

Dijkstra's single source shortest path algorithm when run from vertex a in the below graph, computes the correct shortest path distance to



A only vertex a

B only vertices a, e, f, g, h

only vertices a, b, c, d

all the vertices

Graph Shortest Paths
Discuss it

Question 3 Explanation:
Dijkstra's single source shortest path is not guaranteed to work for graphs with negative weight edges, but it works for the given graph. Let us see... Let us run the 1st pass b 1 b is minimum, so shortest distance to b is 1. After 1st pass, distances are c 3, e -2. e is minimum, so shortest distance to e is -2 After 2nd pass, distances are c 3, f 0. f is minimum, so shortest distance to f is 0 After 3rd pass, distances are c 3, g 3. Both are same, let us take g. so shortest distance to g is 3. After 4th pass,

distances are c 3, h 5 c is minimum, so shortest distance to c is 3 After 5th pass, distances are h -2 h is minimum, so shortest distance to h is -2

Question 7

Wrong

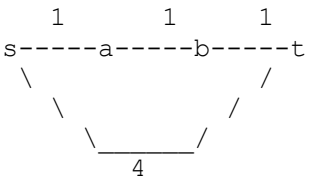
In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same.

Yes

No

Graph Shortest Paths
Discuss it

Question 7 Explanation:
See the following counterexample. There are 4 edges s-a, a-b, b-t and s-t of wights 1, 1, 1 and 4 respectively. The shortest path from s to t is s-a, a-b, b-t. IF we increase weight of every edge by 1, the shortest path changes to s-t.



Question 8

Wrong

Following statement is true or false?

```
If we make following changes to  Dijkstra, then it can be used to find
the longest simple path, assume that the graph is acyclic.

1) Initialize all distances as minus infinite instead of plus infinite.

2) Modify the relax condition in  Dijkstra's algorithm to update distance
  of an adjacent v of the currently considered vertex u only
  if "dist[u]+graph[u][v] > dist[v]". In shortest path algo,
  the sign is opposite.
```

True

False

Graph Shortest Paths
Discuss it

Question 8 Explanation:
In shortest path algo, we pick the minimum distance vertex from the set of vertices for which distance is not finalized yet. And we finalize the distance of the minimum distance vertex. For maximum distance problem, we cannot finalize the distance because there can be a longer path through not yet finalized vertices.

Question 9

Wrong

Which of the following algorithm can be used to **efficiently** calculate single source shortest paths in a Directed Acyclic Graph?

Dijkstra

B Bellman-Ford

Topological Sort

D Strongly Connected Component

Graph Shortest Paths
Discuss it

Question 9 Explanation:
Using Topological Sort, we can find single source shortest paths in O(V+E) time which is the most efficient algorithm. See following for details. Shortest Path in Directed Acyclic Graph

Question 11

Wrong

Is the following statement valid about shortest paths? *Given a graph, suppose we have calculated shortest path from a source to all other vertices. If we modify the graph such that weights of all edges is becomes double of the original weight, then the shortest path remains same only the total weight of path changes.*

True

False

Graph Shortest Paths
Discuss it

Question 11 Explanation:
The shortest path remains same. It is like if we change unit of distance from meter to kilo meter, the shortest paths don't change.

Question 12

Wrong

Match the following

```
Group A                                                  Group B
a) Dijkstra's single shortest path algo                  p) Dynamic
Programming
b) Bellmen Ford's single shortest path algo              q) Backtracking
c) Floyd Warshell's all pair shortest path algo.         r) Greedy Algorithm
```

a-r, b-q, c-p

B a-p, b-p, c-p

a-r, b-p, c-p

D a-p, b-r, c-q

Graph Shortest Paths
Discuss it

Question 12 Explanation:
Dijkstra is a greedy algorithm where we pick the minimum distant vertex from not yet finalized vertices. Bellman Ford and Floyd Warshell both are Dynamic Programming algorithms where we build the shortest paths in bottom up manner.

Question
13

Question 13

Wrong

Is the following statement valid?.

Given a weighted graph where weights of all edges are unique (no two edge have same weights), there is always a unique shortest path from a source to destination in such a graph.

True

False

Graph Shortest Paths
Discuss it

Question 13 Explanation:
There can be more than one paths with same weight. Consider a path with one edge of weight 5 and another path with two edges of weights 2 and 3. Both paths have same weights.

Question
14

Question 14

Correct

Is the following statement valid?.

Given a graph where all edges have positive weights, the shortest paths produced by Dijsktra and Bellman Ford algorithm may be different but path weight would always be same.

True

B False

Graph Shortest Paths
Discuss it

Question 14 Explanation:
Dijkstra and Bellman-Ford both work fine for a graph with all positive weights, but they are different algorithms and may pick different edges for shortest paths.

Question 15

Correct

Which of the following statement(s)is / are correct regarding Bellman-Ford shortest path algorithm?

```
P: Always finds a negative weighted cycle, if one exist s.
Q: Finds whether any negative weighted cycle is reachable
   from the source.
```

A P Only

Q Only

C Both P and Q

D Neither P nor Q

Graph Shortest Paths **GATE-CS-2009**
Discuss it

Question 15 Explanation:
Bellman-Ford shortest path algorithm is a single source shortest path algorithm. So it can only find cycles which are reachable from a given source, not any negative weight cycle. Consider a disconnected where a negative weight cycle is not reachable from the source at all. If there is a negative weight cycle, then it will appear in shortest path as the negative weight cycle will always form a shorter path when iterated through the cycle again and again.

Question 16

Wrong

Let G(V, E) an undirected graph with positive edge weights. Dijkstra's single-source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A O(| V |2)

B O (| E | + | V | log | V |)

O (| V | log | V |)

O ((| E | + | V |) log | V |)

Graph Shortest Paths **GATE-CS-2005**
Discuss it

Question 16 Explanation:
See Time Complexity Section of http://www.geeksforgeeks.org/greedy-algorithms-set-7-dijkstras-algorithm-for-adjacency-list-representation/

Question
17

Question 17

Wrong

Let G = (V, E) be an undirected graph with a subgraph G1 = (V1, El). Weights are assigned to edges of G as follows :

$$w(e) = \begin{cases} 0 \text{ if } e \in E_1 \\ 1 \text{ otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v1 of V1 as the source. Which of the following can always be inferred from the path costs computed?

The number of edges in the shortest paths from v1 to all vertices of G

G1 is connected

C V1 forms a clique in G

D G1 is a tree

Graph Shortest Paths **GATE-CS-2003**
Discuss it

Question 17 Explanation:
When shortest path shortest path from v1 (one of the vertices in V1) is computed. G1 is connected if the distance from v1 to any other vertex in V1 is greater than 0, otherwise G1 is disconnected.

Question 19

Correct

Let G = (V, E) be a simple undirected graph, and s be a particular vertex in it called the source. For x ∈ V, let d(x) denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of d(u) – d(v)?

A -1

B 0

C 1

2

Graph Shortest Paths **GATE-CS-2015 (Set 1)**
Discuss it

Question 19 Explanation:
Note that the given graph is undirected, so an edge (u, v) also means (v, u) is also an edge. Since a shorter path can always be obtained by using edge (u, v) or (v, u), the difference between d(u) and d(v) can not be more than 1.

Question 20

Wrong

Let G be a directed graph whose vertex set is the set of numbers from 1 to 100. There is an edge from a vertex i to a vertex j iff either j = i + 1 or j = 3i. The minimum number of edges in a path in G from vertex 1 to vertex 100 is

A 4

7

23

D 99

Graph Shortest Paths **Gate IT 2005**
Discuss it

Question 20 Explanation:
The task is to find minimum number of edges in a path in G from vertex 1 to vertex 100 such that we can move to either i+1 or 3i from a vertex i.

```
Since the task is to minimize number of edges,
we would prefer to follow 3*i.

Let us follow multiple of 3.

1 => 3 => 9 => 27 => 81, now we can't follow multiple
of 3. So we will have to follow i+1. This solution gives
a long path.

What if we begin from end, and we reduce by 1 if
the value is not multiple of 3, else we divide by 3.
100 => 99 => 33 => 11 => 10 => 9 => 3 => 1

So we need total 7 edges.
```

Question 21

Wrong

Consider the weighted undirected graph with 4 vertices, where the weight of edge {i, j} g is given by the entry W_{ij} in the matrix W

$$W = \begin{bmatrix} 0 & 2 & 8 & 5 \\ 2 & 0 & 5 & 8 \\ 8 & 5 & 0 & x \\ 5 & 8 & x & 0 \end{bmatrix}$$

The largest possible integer value of x, for which at least one shortest path between some pair of

vertices will contain the edge with weight x is _____
Note : This question was asked as Numerical Answer Type.

8

12

C 10

D 11

Graph Shortest Paths **GATE-CS-2016 (Set 1)**
Discuss it

Question 21 Explanation:
Let vertices be 0, 1, 2 and 3. x directly connects 2 to 3. The shortest path (excluding x) from 2 to 3 is of weight 12 (2-1-0-3).

Question 23

Wrong

Consider a weighted undirected graph with positive edge weights and let uv be an edge in the graph. It is known that the shortest path from the source vertex s to u has weight 53 and the shortest path from s to v has weight 65. Which one of the following statements is always true?

A weight (u, v) < 12

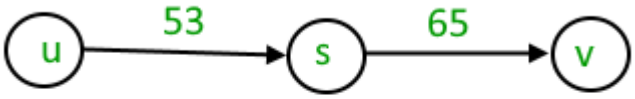
weight (u, v) ≤ 12

C weight (u, v) > 12

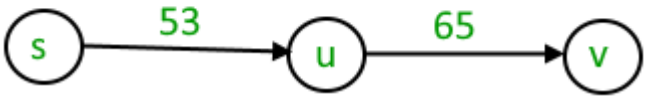
weight (u, v) ≥ 12

Graph Shortest Paths **Graph Theory** **Gate IT 2007**
Discuss it

Question 23 Explanation:



Weight(u,v) >=12



The minimum weight happens when (S,U) + (U,V) = (S,V) Else (S,U) + (U,V) >= (S,V) Given (S,U) = 53, (S,V) = 65 53 + (U,V) >= 63 (U,V) >= 12.
This solution is contributed by **Anil Saikrishna Devarasetty**

Question 27

Correct

Which of the following is application of Breath First Search on the graph?

A Finding diameter of the graph

B Finding bipartite graph

Both (a) and (b)

D None of the above

Graph Shortest Paths **ISRO CS 2018**
Discuss it

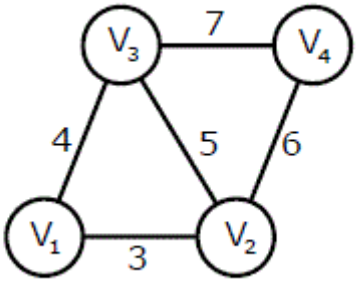
Question 27 Explanation:
BFS is used to Find the diameter of the graph and to test whether a graph is bipartite or not. BFS has many other applications also. Refer: Applications of Breadth First Traversal Option (C) is correct.

Question 1

Correct

An undirected graph G(V, E) contains n (n > 2) nodes named v1 , v2 ,….vn. Two nodes vi , vj are connected if and only if 0 < |i – j| <= 2. Each edge (vi, vj) is assigned a weight i + j. A sample graph

with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a



graph with n nodes? (GATE CS 2011)

A 1/12(11n^2 – 5n)

n^2 – n + 1

C 6n – 11

D 2n + 1

Graph Minimum Spanning Tree
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/data-structures-and-algorithms-set-27/ for explanation.

Question 3

Wrong

Consider a complete undirected graph with vertex set {0, 1, 2, 3, 4}. Entry Wij in the matrix W below is the weight of the edge {i, j}. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T? (GATE CS 2010)

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

A 7

B 8

9

10

Graph Minimum Spanning Tree
Discuss it

Question 3 Explanation:
To get the minimum spanning tree with vertex 0 as leaf, first remove 0th row and 0th column and then get the minimum spanning tree (MST) of the remaining graph. Once we have MST of the remaining graph, connect the MST to vertex 0 with the edge with minimum weight (we have two options as there are two 1s in 0th row).

Question 4

Wrong

In the graph given in above question question, what is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

A 7

8

C 9

10

Graph Minimum Spanning Tree
Discuss it

Question 4 Explanation:
Path: 1 -> 0 -> 4 -> 2 Weight: 1 + 4 + 3

Question 5

Correct

An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0‘s and (ii) non-diagonal elements are 1‘s. which one of the following is TRUE?

A Graph G has no minimum spanning tree (MST)

B Graph G has a unique MST of cost n-1

Graph G has multiple distinct MSTs, each of cost n-1

D Graph G has multiple spanning trees of different costs

Graph Minimum Spanning Tree
Discuss it

Question 5 Explanation:
See Question 2 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-22/

Question 7

Wrong

Let G be an undirected connected graph with distinct edge weight. Let emax be the edge with maximum weight and emin the edge with minimum weight. Which of the following statements is false? (GATE CS 2000)

A Every minimum spanning tree of G must contain emin

If emax is in a minimum spanning tree, then its removal must disconnect G

No minimum spanning tree contains emax

D G has a unique minimum spanning tree

Graph Minimum Spanning Tree
Discuss it

Question 7 Explanation:
(a) and (b) are always true. (c) is false because (b) is true. (d) is true because all edge weights are distinct for G.

Question 8

Correct

Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is: (GATE CS 2006)

A n — 1

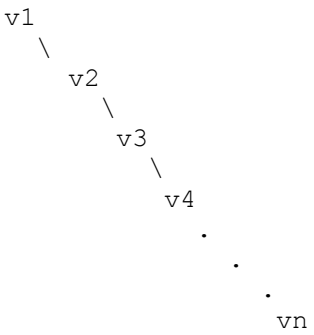
2n — 2

C nC2

D 2

Graph Minimum Spanning Tree
Discuss it

Question 8 Explanation:
Minimum spanning tree of such a graph is



Weight of the minimum spanning tree = 2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| .... + 2| n - (n-1) | = 2n - 2

Question 9

Wrong

Let G be a weighted graph with edge weights greater than one and G'be the graph constructed by squaring the weights of edges in G. Let T and T' be the minimum spanning trees of G and G', respectively, with total weights t and t'. Which of the following statements is TRUE?

A T' = T with total weight t' = t^2

T' = T with total weight t' < t^2

C T' != T but total weight t' = t^2

None of the above

GATE CS 2012 **Graph Minimum Spanning Tree**
Discuss it

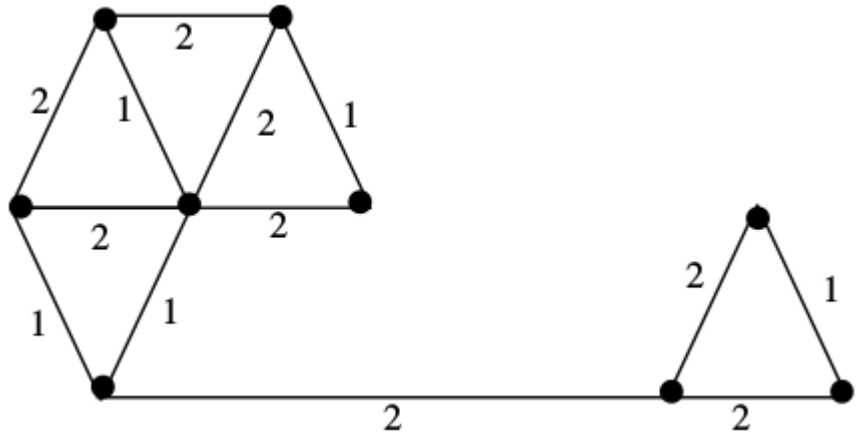
Question 9 Explanation:
Squaring the weights of the edges in a weighted graph will not change the minimum spanning tree. Assume the opposite to obtain a contradiction. If the minimum spanning tree changes then at least one edge from the old graph G in the old minimum spanning tree T must be replaced by a new edge in tree T' from the graph G' with squared edge weights. The new edge from G' must have a lower weight than the edge from G. This implies that there exists some weights C1 and C2 such that C1 < C2 and C12 >= C22. This is a contradiction.
Source: http://www.cs.nyu.edu/courses/spring06/V22.0310-001/hw3.htm Sums of squares of two or more numbers is always smaller than square of sum. Example: 2^2 + 2^2 < (4)^2 But

```
there is one counter example when the graph has only one edge.
         In that case, the two values are same.
```

Question 11

Wrong

The number of distinct minimum spanning trees for the weighted graph below is ____



4

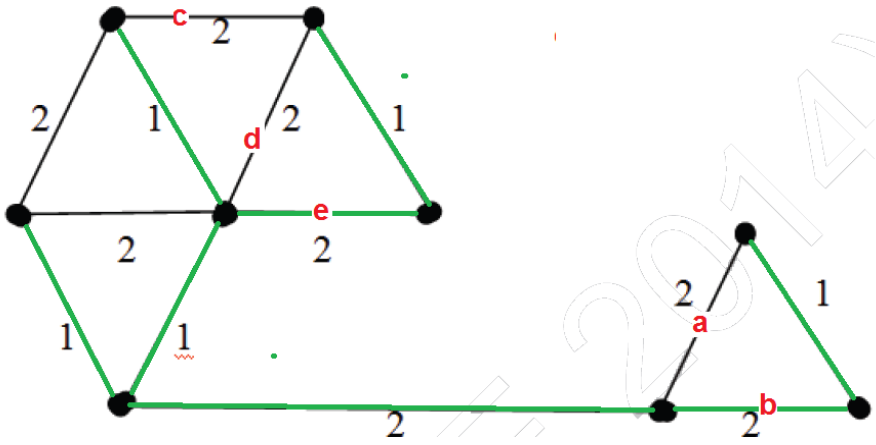
B 5

6

D 7

[Graph Minimum Spanning Tree](#) [GATE-CS-2014-(Set-2)](#)
[Discuss it](#)

Question 11 Explanation:
Below diagram shows a minimum spanning tree. Highlighted (in green) are the edges picked to make the MST. In the right side of MST, we could either pick edge 'a' or 'b'. In the left side, we could either pick 'c' or 'd' or 'e' in MST. There are 2 options for one edge to be picked and 3 options for another edge to be picked. Therefore, total 2*3 possible MSTs.



Question 12

Correct

Let s and t be two vertices in a undirected graph G + (V, E) having distinct positive edge weights. Let [X, Y] be a partition of V such that s ∈ X and t ∈ Y. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y The edge e must definitely belong to:

the minimum weighted spanning tree of G

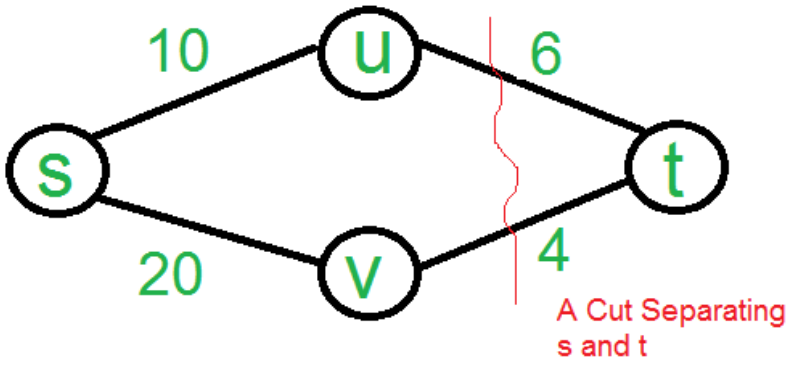
B the weighted shortest path from s to t

C each path from s to t

D the weighted longest path from s to t

Graph Minimum Spanning Tree **GATE-CS-2005**
Discuss it

Question 12 Explanation:
The minimum weight edge on any s-t cut is always part of MST. This is called Cut Property. This is the idea used in Prim's algorithm. The minimum weight cut edge is always a minimum spanning tree edge. **Why B (the weighted shortest path from s to t) is not an answer?** See below example, edge 4 (lightest in highlighted red cut from s to t) is not part of shortest path.



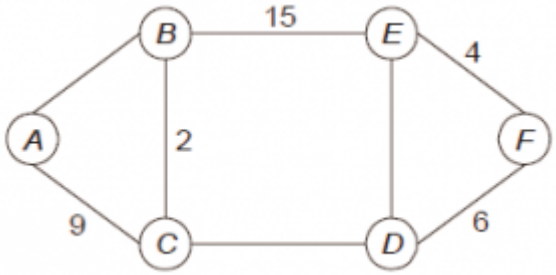
Question
13

Question 15

Wrong

The graph shown below 8 edges with distinct integer edge weights. The minimum spanning tree (MST) is of weight 36 and contains the edges: {(A, C), (B, C), (B, E), (E, F), (D, F)}. The edge weights of only those edges which are in the MST are given in the figure shown below. The minimum possible

sum of weights of all 8 edges of this graph is _____.



66

69

C 68

D 70

Graph Minimum Spanning Tree GATE-CS-2015 (Set 1)
Discuss it

Question 15 Explanation:
In every cycle, the weight of an edge that is not part of MST must by greater than or equal to weights of other edges which are part of MST. Since all edge weights are distinct, the weight must be greater. So the minimum possible weight of ED is 7, minimum possible weight of CD is 16 and minimum possible weight of AB is 10. Therefore minimum possible sum of weights is 69.

Question 16

Wrong

Let G be connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____.

1000

995

C 2000

D 1995

Graph Minimum Spanning Tree GATE-CS-2015 (Set 3)
Discuss it

Question 16 Explanation:
Since there are 100 vertices, there must be 99 edges in Minimum Spanning Tree (MST). When weight of every edge is increased by 5, the increment in weight of MST is = 99 * 5 = 495 So new weight of MST is 500 + 495 which is 995

Question 17

Wrong

Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?

```
P: Minimum spanning tree of G does not change
Q: Shortest path between any pair of vertices does not change
```

P only

B Q only

C Neither P nor Q

Both P and Q

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 17 Explanation:
The shortest path may change. The reason is, there may be different number of edges in different paths from s to t. For example, let shortest path be of weight 15 and has 5 edges. Let there be another path with 2 edges and total weight 25. The weight of the shortest path is increased by 5*10 and becomes 15 + 50. Weight of the other path is increased by 2*10 and becomes 25 + 20. So the shortest path changes to the other path with weight as 45. The Minimum Spanning Tree doesn't change. Remember the Kruskal's algorithm where we sort the edges first. IF we increase all weights, then order of edges won't change.

Question
18

Question 18

Wrong

Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being 1, 2, 3, 4, 5, and 6. The maximum possible weight that a minimum weight spanning tree of G can have is. [This Question was originally a Fill-in-the-Blanks question]

6

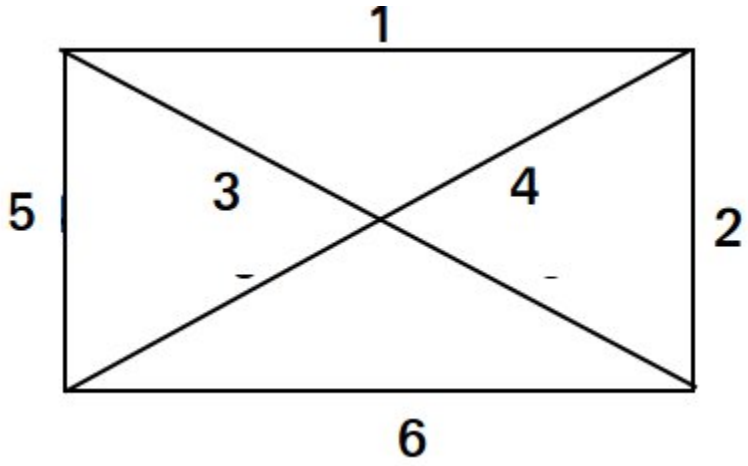
7

C 8

D 9

Graph Minimum Spanning Tree **GATE-CS-2016 (Set 1)**
Discuss it

Question 18 Explanation:
One graph that has maximum possible weight of spanning tree



Question
19

Question 19

Correct

G = (V, E) is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G. Which of the following statements about the minimum spanning trees (MSTs) of G is/are TRUE

```
I.  If e is the lightest edge of some cycle in G,
    then every MST of G includes e
II. If e is the heaviest edge of some cycle in G,
    then every MST of G excludes e
```

A I only

II only

C both I and II

D neither I nor II

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 19 Explanation:
I is NOT true. Let G=(V, E) be a rectangular graph where V = {a, b, c, d} and E = {ab, bc, cd, da, ac}. Let the edges have weights: ab = 1, bc = 2, cd = 4, da = 5, ac = 3. Then, clearly, ac is the lightest edge of the cycle cdac, however, the MST abcd with cost 7 (= ab + bc + cd) does not include it. Let the edges have weights: ab = 6, bc - 7, cd = 4, da = 5, ac = 3. Then, again, ac is the lightest edge of the cycle cdac, and, the MST bacd with cost 13 (= ba + ac + cd) includes it. So, the MSTs of G may or may not include the lightest edge. **II is true** Let the heavies edge be e. Suppose the minimum spanning tree which contains e. If we add one more edge to the spanning tree we will create a cycle. Suppose we add edge e' to the spanning tree which generated cycle C. We can reduce the cost of the minimum spanning tree if we choose an edge other than e from C for removal which implies that e must not be in minimum spanning tree and we get a contradiction. Source:
http://www.ece.northwestern.edu/~dda902/336/hw5-sol.pdf

Question 20

Correct

What is the largest integer m such that every simple connected graph with n vertices and n edges contains at least m different spanning trees?

A 1

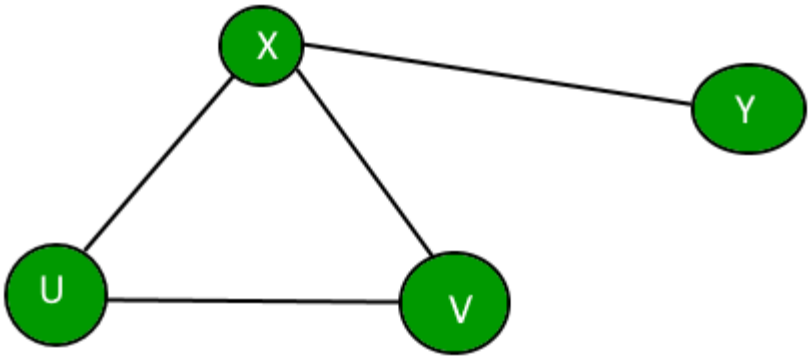
B 2

3

D n

Graph Graph Minimum Spanning Tree Gate IT 2007
Discuss it

Question 20 Explanation:
A graph is connected iff all nodes can be traversed from each node. For a graph with n nodes, there will be n-1 minimum number of edges.
Given that there are n edges, that means a cycle is there in the graph.
The simplex graph with these conditions may be:



Now we can make a different spanning tree by removing one edge from the cycle, one at a time. Minimum cycle length can be 3, So, there must be atleast 3 spanning trees in any such Graph.

Question 21

Wrong

If Kruskal's algorithm is used for finding a minimum spanning tree of a weighted graph G with n vertices and m edges and edge weights are already given in a sorted list, then, What will be the time complexity to compute the minimum cost spanning tree given that union and find operations take amortized O(1) ?

A O(m logn)

B O(n)

O(m)

O(n logm)

Graph Minimum Spanning Tree **GATE 2017 Mock**
Discuss it

Question 21 Explanation:
O(m) as you are already given edge weights in sorted order. You just have to pick the edges in the increasing order and add it to the current spanning set if its addition does not result in a cycle else throw it away.